

SQL_DEFINITIONS_COMPLETE

- [Helix Cluster OS — Complete SQL Definitions](#)
 - [Table of Contents](#)
- [Part 1: PostgreSQL Schema](#)
 - [1.1 nodes \(Migration 001\)](#)
 - [CREATE TABLE Statement](#)
 - [Column Descriptions](#)
 - [Status Enum](#)
 - [Role Enum](#)
 - [Index Definitions](#)
 - [Foreign Key Relationships](#)
 - [Trigger Definitions](#)
 - [Sample Data Pattern](#)
 - [1.2 gpu_devices \(Migration 002\)](#)
 - [CREATE TABLE Statement](#)
 - [Column Descriptions](#)
 - [Status Enum](#)
 - [Index Definitions](#)
 - [Foreign Key Relationships](#)
 - [Trigger Definitions](#)
 - [Sample Data Pattern](#)
 - [1.3 sessions \(Migration 003\)](#)
 - [CREATE TABLE Statement](#)
 - [Column Descriptions](#)
 - [Status Enum](#)
 - [Index Definitions](#)
 - [Foreign Key Relationships](#)
 - [Trigger Definitions](#)
 - [1.4 session_windows \(Migration 004\)](#)
 - [CREATE TABLE Statement](#)
 - [Column Descriptions](#)
 - [Index Definitions](#)
 - [Foreign Key Relationships](#)
 - [1.5 session_panes \(Migration 005\)](#)
 - [CREATE TABLE Statement](#)
 - [Column Descriptions](#)
 - [Status Enum](#)
 - [Index Definitions](#)
 - [Foreign Key Relationships](#)
 - [1.6 reservations \(Migration 006\)](#)
 - [CREATE TABLE Statement](#)
 - [Column Descriptions](#)
 - [Index Definitions](#)
 - [Foreign Key Relationships](#)
 - [Trigger Definitions](#)
 - [1.7 scheduling_queue \(Migration 007\)](#)
 - [CREATE TABLE Statement](#)

- [Column Descriptions](#)
 - [Index Definitions](#)
 - [Trigger Definitions](#)
- [1.8 health_snapshots \(Migration 008\)](#)
 - [CREATE TABLE Statement](#)
 - [Column Descriptions](#)
 - [Index Definitions](#)
- [1.9 ilm_advisories \(Migration 009\)](#)
 - [CREATE TABLE Statement](#)
 - [Column Descriptions](#)
 - [Index Definitions](#)
 - [Trigger Definitions](#)
- [1.10 audit_log \(Migration 010\)](#)
 - [CREATE TABLE Statement](#)
 - [Partition Definitions](#)
 - [Column Descriptions](#)
 - [Index Definitions](#)
- [1.11 build_jobs \(Migration 011\)](#)
 - [CREATE TABLE Statement](#)
 - [Index Definitions](#)
 - [Foreign Key Relationships](#)
 - [Trigger Definitions](#)
- [1.12 build_artifacts \(Migration 012\)](#)
 - [CREATE TABLE Statement](#)
 - [Index Definitions](#)
 - [Foreign Key Relationships](#)
- [1.13 users \(Migration 013\)](#)
 - [CREATE TABLE Statement](#)
 - [Column Descriptions](#)
 - [Index Definitions](#)
 - [Trigger Definitions](#)
- [1.14 migration_history \(Migration 014\)](#)
 - [CREATE TABLE Statement](#)
 - [Column Descriptions](#)
 - [Index Definitions](#)
 - [Constraint Notes](#)
- [1.15 Triggers and Functions \(Migration 015\)](#)
 - [Function: helix_update_updated_at_column\(\)](#)
 - [Function: helix_audit_trigger\(\)](#)
 - [Complete Trigger Map](#)
 - [network_policies \(Migration 015\)](#)
 - [cluster_config \(Migration 015\)](#)
 - [Seed Data](#)
- [1.16 Complete Schema Summary](#)
 - [Entity-Relationship Diagram \(Text\)](#)
 - [Table Count](#)
 - [Total Index Count: 53](#)
- [Part 2: DQLite Schema](#)
 - [2.1 Complete DQLite Schema](#)
 - [2.2 DQLite Index Definitions](#)
 - [2.3 DQLite vs PostgreSQL Comparison](#)

- [Part 3: SQLite Registry Schema](#)
 - [3.1 items](#)
 - [Column Descriptions](#)
 - [3.2 item_history](#)
 - [3.3 document_sources](#)
 - [3.4 meta](#)
 - [3.5 Registry Indexes](#)
- [Part 4: etcd Key Schema](#)
 - [4.1 Key Hierarchy](#)
 - [4.2 Lease Usage](#)
 - [4.3 Watch Patterns](#)
 - [4.4 Concurrency Control](#)
 - [4.5 Data Size Guidelines](#)
- [Part 5: Redis Key Schema](#)
 - [5.1 Session State \(CRDT-synced, short TTL\)](#)
 - [5.2 Node Hot Data](#)
 - [5.3 GPU Status](#)
 - [5.4 Cache Aggregates](#)
 - [5.5 Rate Limiting](#)
 - [5.6 Pub/Sub Channels](#)
 - [5.7 Scheduler State](#)
 - [5.8 Build System](#)
 - [5.9 TTL Strategy Summary](#)
 - [5.10 Consistency Notes](#)
- [Part 6: Schema Drift Analysis](#)
 - [6.1 HXC-1639: Drift Guard Mechanism](#)
 - [How It Works](#)
 - [Regeneration](#)
 - [Drift Guard Tests](#)
 - [Structural Verification](#)
 - [6.2 Known Divergence Points](#)
 - [6.3 HXC-1639 Remediation Plan](#)
 - [Phase 1: Immediate \(Completed\)](#)
 - [Phase 2: Seed Data Alignment](#)
 - [Phase 3: Automation](#)
 - [Phase 4: Partition Automation](#)
- [Part 7: Recommended Schema Improvements](#)
 - [7.1 Missing Indexes](#)
 - [High Priority](#)
 - [Medium Priority](#)
 - [7.2 Partitioning Recommendations](#)
 - [health_snapshots — Partition by RANGE \(recorded_at\)](#)
 - [scheduling_queue — Partition by RANGE \(created_at\)](#)
 - [llm_advisories — Partition by RANGE \(created_at\)](#)
 - [7.3 Data Retention Policies](#)
 - [Retention Job Template](#)
 - [7.4 Backup and Recovery Procedures](#)
 - [PostgreSQL Backup](#)
 - [DQLite Backup](#)
 - [SQLite Registry Backup](#)
 - [etcd Backup](#)
 - [Redis Backup](#)

- [Recovery Procedures](#)
- [7.5 Additional Recommendations](#)
 - [7.5.1 ENUM Types vs CHECK Constraints](#)
 - [7.5.2 JSONB Schema Validation](#)
 - [7.5.3 Created-By / Updated-By Tracking](#)
 - [7.5.4 Soft Delete for Critical Tables](#)
 - [7.5.5 Connection Pooling Configuration](#)

Helix Cluster OS — Complete SQL Definitions

Document ID: SQL-DEFS-001 **Revision:** 1 **Date:** 2026-03-05
Authority: HXC-1639 (drift guard), Constitution §11.4.106
Scope: All persistent schema definitions across PostgreSQL, DQLite, SQLite Registry, etcd, and Redis

Table of Contents

1. [Part 1: PostgreSQL Schema \(Migrations 001-015\)](#)
 2. [Part 2: DQLite Schema](#)
 3. [Part 3: SQLite Registry Schema](#)
 4. [Part 4: etcd Key Schema](#)
 5. [Part 5: Redis Key Schema](#)
 6. [Part 6: Schema Drift Analysis](#)
 7. [Part 7: Recommended Schema Improvements](#)
-

Part 1: PostgreSQL Schema

The authoritative PostgreSQL 16+ schema is defined by two representations that must remain in lockstep:

1. **The golang-migrate chain** (`001_*.up.sql` through `015_*.up.sql`) — the single canonical source (HXC-1639).
2. **The consolidated artifact** (`0001_primary_schema.sql`) — the in-order concatenation of the chain, produced by `migrations/postgresql/.gen_schema.py` . Applied by `internal/schema.ApplyPrimarySchema()` .

The drift-guard test `internal/schema/drift_guard_test.go` fails the build if these two sources ever diverge.

1.1 nodes (Migration 001)

The `nodes` table is the central registry for every physical or virtual machine in the Helix cluster. It stores hardware capabilities, WireGuard

identity, SPIFFE identity, operational status, and spatial metadata (region, labels).

CREATE TABLE Statement

```
CREATE TABLE IF NOT EXISTS nodes (
  id                UUID PRIMARY KEY DEFAULT
    gen_random_uuid(),
  hostname          VARCHAR(255) NOT NULL,
  ip_addresses      INET[] NOT NULL,
  wg_pubkey         TEXT NOT NULL,
  spiffe_id         TEXT NOT NULL,
  status            VARCHAR(20) NOT NULL DEFAULT 'JOINING',
  role              VARCHAR(20) NOT NULL DEFAULT 'WORKER',
  cpu_arch          VARCHAR(20) NOT NULL,
  cpu_cores         INT NOT NULL,
  cpu_threads       INT NOT NULL,
  memory_bytes      BIGINT NOT NULL,
  gpu_count         INT NOT NULL DEFAULT 0,
  storage_bytes     BIGINT NOT NULL,
  labels            JSONB NOT NULL DEFAULT '{}',
  region            VARCHAR(100),
  version           VARCHAR(50) NOT NULL,
  joined_at         TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  last_seen         TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  left_at           TIMESTAMPTZ,
  created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),

  CONSTRAINT nodes_wg_pubkey_unique    UNIQUE (wg_pubkey),
  CONSTRAINT nodes_spiffe_id_unique    UNIQUE (spiffe_id),
  CONSTRAINT nodes_cpu_cores_positive  CHECK (cpu_cores >
    0),
  CONSTRAINT nodes_cpu_threads_ge_cores CHECK
    (cpu_threads >= cpu_cores),
  CONSTRAINT nodes_memory_positive     CHECK
    (memory_bytes > 0),
  CONSTRAINT nodes_gpu_count_nonneg    CHECK (gpu_count
    >= 0),
  CONSTRAINT nodes_storage_positive    CHECK
    (storage_bytes > 0),
  CONSTRAINT nodes_status_valid        CHECK (status IN (
    'JOINING', 'READY', 'DRAINING', 'OFFLINE',
    'MAINTENANCE', 'EVICTED'
  )),
  CONSTRAINT nodes_role_valid          CHECK (role IN (
    'WORKER', 'CONTROL_PLANE', 'GPU_WORKER', 'EDGE',
    'OBSERVER'
```

```
))  
);
```

Column Descriptions

Column	Type	Nullable	Default	Description
<code>id</code>	UUID	NO	<code>gen_random_uuid()</code>	Primary key; globally unique node identifier
<code>hostname</code>	VARCHAR(255)	NO	—	Human-readable hostname (must be unique in practice; enforced at application level)
<code>ip_addresses</code>	INET[]	NO	—	Array of network addresses (WireGuard internal + physical); PostgreSQL INET type enables CIDR operations
<code>wg_pubkey</code>	TEXT	NO	—	WireGuard public key (Base64); UNIQUE constraint ensures one identity per key
<code>spiffe_id</code>	TEXT	NO	—	SPIFFE identity URI (e.g., <code>spiffe://helix.cluster.nodes/alpha</code>); UNIQUE constraint ties node to identity backbone
<code>status</code>	VARCHAR(20)	NO	<code>'JOINING'</code>	Node lifecycle state; see status enum below

Column	Type	Nullable	Default	Description
role	VARCHAR(20)	NO	'WORKER'	Node role determining scheduling behavior
cpu_arch	VARCHAR(20)	NO	—	CPU architecture string: x86_64 , arm64 , aarch64 , etc.
cpu_cores	INT	NO	—	Physical CPU cores; must be > 0
cpu_threads	INT	NO	—	Logical CPU threads (SMT/ hyperthreading); must be >= cpu_cores
memory_bytes	BIGINT	NO	—	Total RAM in bytes; must be > 0
gpu_count	INT	NO	0	Number of GPU devices attached; >= 0
storage_bytes	BIGINT	NO	—	Total local storage in bytes; must be > 0
labels	JSONB	NO	'{}'	Arbitrary key-value labels for scheduling constraints (GIN-indexed)
region	VARCHAR(100)	YES	NULL	Geographic/ datacenter region for placement decisions
version	VARCHAR(50)	NO	—	Running Helix agent version string (semver)
joined_at	TIMESTAMPTZ	NO	NOW()	Timestamp when node first joined

Column	Type	Nullable	Default	Description
				joined the cluster
last_seen	TIMESTAMPTZ	NO	NOW()	Last heartbeat received; used for failure detection
left_at	TIMESTAMPTZ	YES	NULL	Timestamp when node cleanly left the cluster; NULL if still active
created_at	TIMESTAMPTZ	NO	NOW()	Row creation timestamp
updated_at	TIMESTAMPTZ	NO	NOW()	Row last-update timestamp (auto-bumped by trigger)

Status Enum

Status	Description
JOINING	Node is in the process of joining the cluster; not yet ready for work
READY	Node is fully operational and accepting workloads
DRAINING	Node is being gracefully drained; no new sessions scheduled; existing sessions being migrated
OFFLINE	Node is unreachable; heartbeat missed; may be temporary or permanent
MAINTENANCE	Node is under planned maintenance; not accepting workloads
EVICTED	Node has been forcibly removed from the cluster (security or policy violation)

Role Enum

Role	Description
WORKER	General compute node; runs sessions and build jobs
CONTROL_PLANE	Runs cluster control services (scheduler, gateway, etc.)
GPU_WORKER	GPU-optimized node; preferred for GPU-intensive workloads
EDGE	Edge device (SBC, console); limited compute; used for I/O and lightweight tasks
OBSERVER	Read-only node; monitors cluster but does not run workloads

Index Definitions

```
CREATE INDEX IF NOT EXISTS idx_nodes_status ON
nodes(status);
CREATE INDEX IF NOT EXISTS idx_nodes_role ON
nodes(role);
CREATE INDEX IF NOT EXISTS idx_nodes_region ON
nodes(region);
CREATE INDEX IF NOT EXISTS idx_nodes_labels ON nodes
USING GIN(labels);
CREATE INDEX IF NOT EXISTS idx_nodes_last_seen ON
nodes(last_seen);
CREATE INDEX IF NOT EXISTS idx_nodes_hostname ON
nodes(hostname);
```

Index	Type	Purpose
idx_nodes_status	B-tree	Fast lookup of nodes by operational status (e.g., all READY nodes)
idx_nodes_role	B-tree	Filter nodes by role for role-aware scheduling queries
idx_nodes_region	B-tree	Region-based placement queries
idx_nodes_labels	GIN	JSONB containment queries: <code>labels @></code>

Index	Type	Purpose
		'{"gpu_vendor": "nvidia"}'
idx_nodes_last_seen	B-tree	Failure detection: find nodes whose heartbeat is stale
idx_nodes_hostname	B-tree	Human-facing lookup by hostname

Foreign Key Relationships

The `nodes` table is a parent table referenced by: -
`gpu_devices.node_id` → `nodes.id` (ON DELETE CASCADE) -
`sessions.node_id` → `nodes.id` - `session_panes.node_id` → `nodes.id` - `reservations.node_id` → `nodes.id` (ON DELETE CASCADE) - `health_snapshots.node_id` → `nodes.id` (ON DELETE CASCADE) - `migration_history.source_node` → `nodes.id` - `migration_history.target_node` → `nodes.id` - `build_jobs.node_id` → `nodes.id`

Trigger Definitions

Trigger	Event	Timing	Function
helix_nodes_updated_at	UPDATE	BEFORE	helix_update updated at column — auto-sets updated_at = NOW()
helix_nodes_audit	INSERT, UPDATE, DELETE	AFTER	helix_audit_trigger() — writes immutable audit record

Sample Data Pattern

```
INSERT INTO nodes (id, hostname, ip_addresses, wg_pubkey,
    spiffe_id, status, role,
    cpu_arch, cpu_cores, cpu_threads, memory_bytes,
    gpu_count, storage_bytes,
    labels, region, version)
VALUES
    ('a1b2c3d4-e5f6-7890-abcd-ef1234567890', 'node-alpha',
    ARRAY['192.168.1.10'::INET, '10.0.0.10'::INET],
    'alpha_wg_pubkey_1234567890abcdef',
    'spiffe://helix.cluster/nodes/alpha',
    'READY', 'GPU_WORKER', 'x86_64', 16, 32, 68719476736,
    1, 2199023255552,
```

```
'{"rack": "A1", "zone": "us-east-1a", "gpu_vendor":  
  "nvidia"}'::JSONB,  
'us-east-1', '1.0.0-alpha');
```

1.2 gpu_devices (Migration 002)

The `gpu_devices` table catalogs every GPU accelerator attached to cluster nodes. Each GPU is independently tracked for allocation, status, and capability metadata.

CREATE TABLE Statement

```
CREATE TABLE IF NOT EXISTS gpu_devices (  
  id                UUID PRIMARY KEY DEFAULT  
    gen_random_uuid(),  
  node_id           UUID NOT NULL REFERENCES nodes(id) ON  
    DELETE CASCADE,  
  vendor            VARCHAR(20) NOT NULL,  
  model             VARCHAR(100) NOT NULL,  
  driver_version    VARCHAR(50) NOT NULL,  
  api               VARCHAR(20) NOT NULL,  
  api_version       VARCHAR(20) NOT NULL,  
  total_memory      BIGINT NOT NULL,  
  compute_units     INT NOT NULL,  
  features          TEXT[] NOT NULL DEFAULT '{}',  
  attributes        JSONB NOT NULL DEFAULT '{}',  
  status            VARCHAR(20) NOT NULL DEFAULT  
    'AVAILABLE',  
  allocated_to      UUID,  
  created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  
  CONSTRAINT gpu_vendor_valid CHECK (vendor IN  
    ('NVIDIA', 'AMD', 'INTEL', 'APPLE', 'QUALCOMM',  
    'OTHER')),  
  CONSTRAINT gpu_api_valid CHECK (api IN ('CUDA',  
    'ROCm', 'Metal', 'Vulkan', 'OpenCL', 'OTHER')),  
  CONSTRAINT gpu_status_valid CHECK (status IN  
    ('AVAILABLE', 'IN_USE', 'RESERVED', 'OFFLINE',  
    'FAULT')),  
  CONSTRAINT gpu_mem_positive CHECK (total_memory > 0),  
  CONSTRAINT gpu_cu_positive CHECK (compute_units > 0)  
);
```

Column Descriptions

Column	Type	Nullable	Default	Description
id	UUID	NO	gen_random_uuid()	Primary key; globally unique GPU identifier
node_id	UUID	NO	—	Foreign key to nodes.id; CASCADE delete removes GPUs when node is deleted
vendor	VARCHAR(20)	NO	—	GPU vendor: NVIDIA, AMD, INTEL, APPLE, QUALCOMM, OTHER
model	VARCHAR(100)	NO	—	GPU model name (e.g., “GeForce RTX 4080”, “M3 Pro 18-Core GPU”)
driver_version	VARCHAR(50)	NO	—	Installed driver runtime version string
api	VARCHAR(20)	NO	—	Primary compute API: CUDA, ROCm, Metal, Vulkan, OpenCL, OTHER
api_version	VARCHAR(20)	NO	—	API version string (e.g., “12.3” for CUDA 12.3)
total_memory	BIGINT	NO	—	Total VRAM in bytes; must be > 0
compute_units	INT	NO	—	Number of compute units (SMs for NVIDIA, CUs for AMD)

Column	Type	Nullable	Default	Description
				AMD, cores for Apple)
features	TEXT[]	NO	'{}'	Array of capability strings: ray tracing, tensor core, dlss, fsr, etc.
attributes	JSONB	NO	'{}'	Extended attributes: PC generation, power limits, MIG profiles, etc.
status	VARCHAR(20)	NO	'AVAILABLE'	GPU operation status; see enum below
allocated_to	UUID	YES	NULL	Session ID that currently holds this GPU; NULL if free
created_at	TIMESTAMPTZ	NO	NOW()	Row creation timestamp
updated_at	TIMESTAMPTZ	NO	NOW()	Last update timestamp (auto-bumped by trigger)

Status Enum

Status	Description
AVAILABLE	GPU is idle and ready for allocation
IN_USE	GPU is allocated to a session (ref in allocated_to)
RESERVED	GPU is reserved for a future high-priority workload
OFFLINE	GPU is administratively taken offline
FAULT	GPU has a hardware or driver fault; not usable

Index Definitions

```
CREATE INDEX IF NOT EXISTS idx_gpu_devices_node_id ON
  gpu_devices(node_id);
CREATE INDEX IF NOT EXISTS idx_gpu_devices_status ON
  gpu_devices(status);
CREATE INDEX IF NOT EXISTS idx_gpu_devices_vendor ON
  gpu_devices(vendor);
CREATE INDEX IF NOT EXISTS idx_gpu_devices_model ON
  gpu_devices(model);
```

Index	Type	Purpose
idx_gpu_devices_node_id	B-tree	Find all GPUs belonging to a specific node
idx_gpu_devices_status	B-tree	Find available GPUs for scheduling
idx_gpu_devices_vendor	B-tree	Filter by vendor (e.g., all NVIDIA GPUs)
idx_gpu_devices_model	B-tree	Filter by model for workload compatibility

Foreign Key Relationships

- gpu_devices.node_id → nodes.id (ON DELETE CASCADE)
- Referenced by: session_panes.gpu_id → gpu_devices.id

Trigger Definitions

Trigger	Event	Timing	Function
helix_gpu_devices_updated_at	UPDATE	BEFORE	helix_update_updated_at_

Sample Data Pattern

```
INSERT INTO gpu_devices (id, node_id, vendor, model,
  driver_version, api, api_version,
  total_memory, compute_units, features, attributes,
  status)
VALUES
  ('1eb576ad-8b69-5f4d-908a-06807e23ca7b', 'a1b2c3d4-
    e5f6-7890-abcd-ef1234567890',
    'NVIDIA', 'GeForce RTX 4080', '545.23.06', 'CUDA',
    '12.3',
```

```
17179869184, 76, ARRAY['ray_tracing', 'dlss',  
    'tensor_cores'],  
'{"pcie": "gen4", "power_limit_w": 320}':JSONB,  
    'AVAILABLE');
```

1.3 sessions (Migration 003)

The `sessions` table tracks interactive and batch compute sessions. Each session represents a user's workspace — a collection of windows and panes running on a specific node, with defined resource requests.

CREATE TABLE Statement

```
CREATE TABLE IF NOT EXISTS sessions (  
    id                UUID PRIMARY KEY DEFAULT  
        gen_random_uuid(),  
    name              VARCHAR(255) NOT NULL,  
    owner             TEXT NOT NULL,  
    status            VARCHAR(20) NOT NULL DEFAULT 'CREATING',  
    mode              VARCHAR(20) NOT NULL DEFAULT  
        'INTERACTIVE',  
    backend           VARCHAR(20) NOT NULL DEFAULT 'TMUX',  
    backend_id        TEXT,  
    node_id           UUID REFERENCES nodes(id),  
    cpu_request        INT NOT NULL DEFAULT 1000,  
    memory_request    BIGINT NOT NULL DEFAULT 1073741824,  
    gpu_request        JSONB DEFAULT NULL,  
    priority           INT NOT NULL DEFAULT 50,  
    labels            JSONB NOT NULL DEFAULT '{}',  
    created_at         TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
    started_at        TIMESTAMPTZ,  
    terminated_at      TIMESTAMPTZ,  
    updated_at         TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  
    CONSTRAINT sessions_status_valid CHECK (status IN (  
        'CREATING', 'RUNNING', 'PAUSED', 'TERMINATING',  
        'TERMINATED', 'FAILED'  
    )),  
    CONSTRAINT sessions_mode_valid CHECK (mode IN (  
        'INTERACTIVE', 'BATCH', 'DAEMON', 'NOTEBOOK'  
    )),  
    CONSTRAINT sessions_backend_valid CHECK (backend IN (  
        'TMUX', 'WASM', 'CONTAINER', 'NATIVE'  
    )),  
    CONSTRAINT sessions_priority_range CHECK (priority >= 0  
        AND priority <= 100),
```

```
CONSTRAINT sessions_cpu_positive CHECK (cpu_request > 0),
CONSTRAINT sessions_mem_positive CHECK
(memory_request > 0)
);
```

Column Descriptions

Column	Type	Nullable	Default	Description
id	UUID	NO	gen_random_uuid()	Primary key; globally unique session identifier
name	VARCHAR(255)	NO	—	Human-readable session name
owner	TEXT	NO	—	SPIFFE ID of the session owner spiffe://helix.cluster/users/alice
status	VARCHAR(20)	NO	'CREATING'	Session lifecycle state
mode	VARCHAR(20)	NO	'INTERACTIVE'	Execution mode: INTERACTIVE, DAEMON, NOT
backend	VARCHAR(20)	NO	'TMUX'	Terminal/computer backend: Tmux, WASM, CONTAINER, NATIVE
backend_id	TEXT	YES	NULL	Backend-specific identifier (e.g. session ID, container ID)
node_id	UUID	YES	NULL	Node where the session is currently executing; NULL during scheduling
cpu_request	INT	NO	1000	CPU request in millicores (1000 = 1 core)
memory_request	BIGINT	NO	1073741824	Memory request in bytes (default)

Column	Type	Nullable	Default	Description
gpu_request	JSONB	YES	NULL	GPU requirements { "gpu count": 1, "gpu memory": 17179869184 }
priority	INT	NO	50	Scheduling priority (0=lowest, 100=highest)
labels	JSONB	NO	'{}'	Arbitrary labels for filtering and scheduling
created_at	TIMESTAMPTZ	NO	NOW()	Session creation timestamp
started_at	TIMESTAMPTZ	YES	NULL	Timestamp when session entered RUNNING state
terminated_at	TIMESTAMPTZ	YES	NULL	Timestamp when session was terminated
updated_at	TIMESTAMPTZ	NO	NOW()	Last update timestamp (always bumped by trigger)

Status Enum

Status	Description
CREATING	Session is being provisioned; resources being allocated
RUNNING	Session is active and processing
PAUSED	Session is temporarily suspended (resource-preserving)
TERMINATING	Session is being shut down; resources being released
TERMINATED	Session has completed (normal or abnormal)
FAILED	Session failed to start or crashed irrecoverably

Index Definitions

```
CREATE INDEX IF NOT EXISTS idx_sessions_owner ON
sessions(owner);
CREATE INDEX IF NOT EXISTS idx_sessions_status ON
sessions(status);
CREATE INDEX IF NOT EXISTS idx_sessions_node_id ON
sessions(node_id);
CREATE INDEX IF NOT EXISTS idx_sessions_mode ON
sessions(mode);
CREATE INDEX IF NOT EXISTS idx_sessions_priority ON
sessions(priority);
CREATE INDEX IF NOT EXISTS idx_sessions_labels ON
sessions USING GIN(labels);
```

Index	Type	Purpose
idx_sessions_owner	B-tree	Find all sessions owned by a specific user
idx_sessions_status	B-tree	Filter sessions by state (e.g., all RUNNING)
idx_sessions_node_id	B-tree	Find all sessions on a given node
idx_sessions_mode	B-tree	Filter by execution mode
idx_sessions_priority	B-tree	Priority-ordered scheduling queries
idx_sessions_labels	GIN	JSONB containment queries for label-based filtering

Foreign Key Relationships

- sessions.node_id → nodes.id
- Referenced by: session windows.session_id, reservations.session_id, migration_history.session_id

Trigger Definitions

Trigger	Event	Timing	Function
helix_sessions_updated_at	UPDATE	BEFORE	helix_update_updated_at_co
helix_sessions_audit		AFTER	helix_audit_trigger()

Trigger	Event	Timing	Function
	INSERT, UPDATE, DELETE		

1.4 session_windows (Migration 004)

The `session_windows` table represents terminal windows within a session. Each window contains one or more panes and has an associated layout and CRDT state for collaborative editing.

CREATE TABLE Statement

```
CREATE TABLE IF NOT EXISTS session_windows (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  session_id  UUID NOT NULL REFERENCES sessions(id) ON
              DELETE CASCADE,
  name        VARCHAR(255) NOT NULL,
  layout      VARCHAR(50) NOT NULL DEFAULT 'tiled',
  active      BOOLEAN NOT NULL DEFAULT FALSE,
  crdt_state  JSONB DEFAULT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),

  CONSTRAINT sw_layout_valid CHECK (layout IN (
    'tiled', 'even-horizontal', 'even-vertical', 'main-
    horizontal', 'main-vertical'
  ))
);
```

Column Descriptions

Column	Type	Nullable	Default	Description
<code>id</code>	UUID	NO	<code>gen_random_uuid()</code>	Primary key; window identifier
<code>session_id</code>	UUID	NO	—	Parent session; CASCADE delete removes windows when session is deleted

Column	Type	Nullable	Default	Description
name	VARCHAR(255)	NO	—	Window name (e.g., “editor”, “logs”)
layout	VARCHAR(50)	NO	'tiled'	Tiling layout algorithm
active	BOOLEAN	NO	FALSE	Whether this is the currently focused window
crdt_state	JSONB	YES	NULL	CRDT merge state for real-time collaborative layout editing
created_at	TIMESTAMPTZ	NO	NOW()	Window creation timestamp

Index Definitions

```
CREATE INDEX IF NOT EXISTS idx_session_windows_session_id
ON session_windows(session_id);
CREATE INDEX IF NOT EXISTS idx_session_windows_active
ON session_windows(active);
```

Foreign Key Relationships

- session_windows.session_id → sessions.id (ON DELETE CASCADE)
- Referenced by: session_panes.window_id

1.5 session_panes (Migration 005)

The session_panes table represents individual terminal panes within a window. Each pane runs a command on a specific node and may be associated with a GPU.

CREATE TABLE Statement

```
CREATE TABLE IF NOT EXISTS session_panes (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  window_id   UUID NOT NULL REFERENCES  
    session_windows(id) ON DELETE CASCADE,  
  node_id     UUID REFERENCES nodes(id),  
  command     TEXT,  
  working_dir TEXT,  
  environment JSONB NOT NULL DEFAULT '{}',  
  cpu_limit   INT,  
  memory_limit BIGINT,  
  gpu_id      UUID REFERENCES gpu_devices(id),  
  status      VARCHAR(20) NOT NULL DEFAULT 'CREATING',  
  crdt_state  JSONB DEFAULT NULL,  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  
  CONSTRAINT sp_status_valid CHECK (status IN (  
    'CREATING', 'RUNNING', 'STOPPED', 'FAILED',  
    'MIGRATING'  
  )),  
  CONSTRAINT sp_cpu_limit_pos CHECK (cpu_limit IS NULL  
    OR cpu_limit > 0),  
  CONSTRAINT sp_mem_limit_pos CHECK (memory_limit IS  
    NULL OR memory_limit > 0)  
);
```

Column Descriptions

Column	Type	Nullable	Default	Description
id	UUID	NO	gen_random_uuid()	Primary key; pane identifier
window_id	UUID	NO	—	Parent window; CASCADE delete
node_id	UUID	YES	NULL	Node where this pane's process is running (may differ from session's node for

Column	Type	Nullable	Default	Description
				distributed sessions)
command	TEXT	YES	NULL	Command being executed in the pane
working_dir	TEXT	YES	NULL	Working directory for the command
environment	JSONB	NO	'{}'	Environment variables as key-value pairs
cpu_limit	INT	YES	NULL	CPU limit in millicores (NULL = unlimited)
memory_limit	BIGINT	YES	NULL	Memory limit in bytes (NULL = unlimited)
gpu_id	UUID	YES	NULL	GPU device allocated to this pane
status	VARCHAR(20)	NO	'CREATING'	Pane lifecycle state
crdt_state	JSONB	YES	NULL	CRDT merge state for collaborative pane state
created_at	TIMESTAMPTZ	NO	NOW()	Pane creation timestamp

Status Enum

Status	Description
CREATING	Pane is being initialized
RUNNING	Pane process is active
STOPPED	Pane process has stopped (normal exit)
FAILED	Pane process failed
MIGRATING	Pane is being migrated to another node

Index Definitions

```
CREATE INDEX IF NOT EXISTS idx_session_panes_window_id ON
    session_panes(window_id);
CREATE INDEX IF NOT EXISTS idx_session_panes_node_id ON
    session_panes(node_id);
CREATE INDEX IF NOT EXISTS idx_session_panes_gpu_id ON
    session_panes(gpu_id);
CREATE INDEX IF NOT EXISTS idx_session_panes_status ON
    session_panes(status);
```

Foreign Key Relationships

- session_panes.window_id → session_windows.id (ON DELETE CASCADE)
- session_panes.node_id → nodes.id
- session_panes.gpu_id → gpu_devices.id

1.6 reservations (Migration 006)

The reservations table tracks resource reservations for sessions. A reservation binds CPU, memory, and GPU resources on a specific node to a session for a defined duration.

CREATE TABLE Statement

```
CREATE TABLE IF NOT EXISTS reservations (
    id                UUID PRIMARY KEY DEFAULT
        gen_random_uuid(),
    session_id        UUID NOT NULL REFERENCES sessions(id)
        ON DELETE CASCADE,
    node_id           UUID NOT NULL REFERENCES nodes(id),
```

```

cpu_millicores    INT NOT NULL,
memory_bytes     BIGINT NOT NULL,
gpu_ids          UUID[] NOT NULL DEFAULT '{}',
status           VARCHAR(20) NOT NULL DEFAULT 'PENDING',
expires_at       TIMESTAMPTZ NOT NULL,
created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),

CONSTRAINT reservations_status_valid CHECK (status IN
(
    'PENDING', 'ACTIVE', 'RELEASED', 'EXPIRED', 'FAILED'
)),
CONSTRAINT reservations_cpu_positive CHECK
(cpu_millicores > 0),
CONSTRAINT reservations_mem_positive CHECK
(memory_bytes > 0),
CONSTRAINT reservations_expires_future CHECK
(expires_at > created_at)
);

```

Column Descriptions

Column	Type	Nullable	Default	Description
id	UUID	NO	gen_random_uuid()	Primary key; reservation identifier
session_id	UUID	NO	—	Session that holds this reservation; CASCADE delete
node_id	UUID	NO	—	Node where resources are reserved
cpu_millicores	INT	NO	—	CPU reservation in millicores (1000 = 1 core)
memory_bytes	BIGINT	NO	—	Memory reservation in bytes
gpu_ids	UUID[]	NO	'{}'	Array of GPU device IDs

Column	Type	Nullable	Default	Description
				reserved for this session
status	VARCHAR(20)	NO	' PENDING '	Reservation lifecycle state
expires_at	TIMESTAMPTZ	NO	—	Reservation expiration time; must be after creation
created_at	TIMESTAMPTZ	NO	NOW()	Reservation creation timestamp
updated_at	TIMESTAMPTZ	NO	NOW()	Last update timestamp (auto-bumped by trigger)

Index Definitions

```
CREATE INDEX IF NOT EXISTS idx_reservations_session_id ON
    reservations(session_id);
CREATE INDEX IF NOT EXISTS idx_reservations_node_id      ON
    reservations(node_id);
CREATE INDEX IF NOT EXISTS idx_reservations_status      ON
    reservations(status);
CREATE INDEX IF NOT EXISTS idx_reservations_expires_at  ON
    reservations(expires_at);
```

Foreign Key Relationships

- reservations.session_id → sessions.id (ON DELETE CASCADE)
- reservations.node_id → nodes.id

Trigger Definitions

Trigger	Event	Timing	Function
helix_reservations_updated_at	UPDATE	BEFORE	helix_update_updated_at

1.7 scheduling_queue (Migration 007)

The `scheduling_queue` table holds pending and active scheduling requests. The Omega-model scheduler processes these entries, matching resource requests to available nodes.

CREATE TABLE Statement

```
CREATE TABLE IF NOT EXISTS scheduling_queue (
  id          UUID PRIMARY KEY DEFAULT
    gen_random_uuid(),
  request     JSONB NOT NULL,
  priority    INT NOT NULL DEFAULT 50,
  status      VARCHAR(20) NOT NULL DEFAULT 'PENDING',
  scheduled_at TIMESTAMPTZ,
  started_at  TIMESTAMPTZ,
  completed_at TIMESTAMPTZ,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),

  CONSTRAINT scheduling_status_valid CHECK (status IN (
    'PENDING', 'SCHEDULING', 'SCHEDULED', 'RUNNING',
    'COMPLETED', 'FAILED', 'CANCELLED'
  )),
  CONSTRAINT scheduling_priority_range CHECK (priority >=
    0 AND priority <= 100)
);
```

Column Descriptions

Column	Type	Nullable	Default	Description
<code>id</code>	UUID	NO	<code>gen_random_uuid()</code>	Primary key; scheduling request identifier
<code>request</code>	JSONB	NO	—	Full scheduling request payload: resource requirements, constraints, preferences
<code>priority</code>	INT	NO	50	

Column	Type	Nullable	Default	Description
				Scheduling priority (0-100)
status	VARCHAR(20)	NO	'PENDING'	Scheduling lifecycle state
scheduled_at	TIMESTAMPTZ	YES	NULL	When the request was assigned to a node
started_at	TIMESTAMPTZ	YES	NULL	When the scheduled workload began execution
completed_at	TIMESTAMPTZ	YES	NULL	When the scheduled workload completed
created_at	TIMESTAMPTZ	NO	NOW()	Request creation timestamp
updated_at	TIMESTAMPTZ	NO	NOW()	Last update timestamp (auto-bumped by trigger)

Index Definitions

```
CREATE INDEX IF NOT EXISTS idx_scheduling_queue_status ON
    scheduling_queue(status);
CREATE INDEX IF NOT EXISTS idx_scheduling_queue_priority ON
    scheduling_queue(priority);
CREATE INDEX IF NOT EXISTS idx_scheduling_queue_created ON
    scheduling_queue(created_at);
```

Trigger Definitions

Trigger	Event	Timing	Function
helix_scheduling_queue_updated_at	UPDATE	BEFORE	helix_update_update

1.8 health_snapshots (Migration 008)

The `health_snapshots` table stores periodic health measurements for each node. Each snapshot contains sub-scores for CPU, memory, disk, network, GPU, temperature, and services, plus predictive analytics and raw metrics.

CREATE TABLE Statement

```
CREATE TABLE IF NOT EXISTS health_snapshots (  
  id BIGSERIAL PRIMARY KEY,  
  node_id UUID NOT NULL REFERENCES nodes(id)  
    ON DELETE CASCADE,  
  overall_score INT NOT NULL,  
  cpu_score INT NOT NULL,  
  memory_score INT NOT NULL,  
  disk_score INT NOT NULL,  
  network_score INT NOT NULL,  
  gpu_score INT NOT NULL,  
  temperature_score INT NOT NULL,  
  services_score INT NOT NULL,  
  predictions JSONB NOT NULL DEFAULT '[]',  
  metrics JSONB NOT NULL DEFAULT '{}',  
  recorded_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  
  CONSTRAINT hs_overall_range CHECK (overall_score  
    BETWEEN 0 AND 100),  
  CONSTRAINT hs_cpu_range CHECK (cpu_score  
    BETWEEN 0 AND 100),  
  CONSTRAINT hs_memory_range CHECK (memory_score  
    BETWEEN 0 AND 100),  
  CONSTRAINT hs_disk_range CHECK (disk_score  
    BETWEEN 0 AND 100),  
  CONSTRAINT hs_network_range CHECK (network_score  
    BETWEEN 0 AND 100),  
  CONSTRAINT hs_gpu_range CHECK (gpu_score  
    BETWEEN 0 AND 100),  
  CONSTRAINT hs_temperature_range CHECK  
    (temperature_score BETWEEN 0 AND 100),  
  CONSTRAINT hs_services_range CHECK (services_score  
    BETWEEN 0 AND 100)  
);
```

Column Descriptions

Column	Type	Nullable	Default	Description
id	BIGSERIAL	NO	auto	Auto-incrementing primary key (high-volume time-series data)
node_id	UUID	NO	—	Node this snapshot belongs to; CASCADE delete
overall_score	INT	NO	—	Weighted aggregate health score (0-100)
cpu_score	INT	NO	—	CPU health sub-score (0-100)
memory_score	INT	NO	—	Memory health sub-score (0-100)
disk_score	INT	NO	—	Disk I/O and capacity sub-score (0-100)
network_score	INT	NO	—	Network latency and throughput sub-score (0-100)
gpu_score	INT	NO	—	GPU health sub-score (0-100)
temperature_score	INT	NO	—	Thermal health sub-score (0-100)
services_score	INT	NO	—	Service availability sub-score (0-100)
predictions	JSONB	NO	'[]'	ML predictions: [{"metric": "cpu temp", "forecast": 72, "horizon": "5m"}]
metrics	JSONB	NO	'{}'	Raw metric values: {"cpu_usage": 0.45, "memory_usage": 0.62}

Column	Type	Nullable	Default	Description
recorded_at	TIMESTAMPTZ	NO	NOW()	When this snapshot was recorded

Index Definitions

```
CREATE INDEX IF NOT EXISTS idx_health_snapshots_node_id
    ON health_snapshots(node_id);
CREATE INDEX IF NOT EXISTS idx_health_snapshots_recorded_at
    ON health_snapshots(recorded_at);
CREATE INDEX IF NOT EXISTS idx_health_snapshots_overall
    ON health_snapshots(overall_score);
```

1.9 llm_advisories (Migration 009)

The `llm_advisories` table stores recommendations generated by the LLM Brain service. Each advisory describes a proposed cluster action, its rationale, confidence level, and risk classification, along with approval workflow state.

CREATE TABLE Statement

```
CREATE TABLE IF NOT EXISTS llm_advisories (
    id                UUID PRIMARY KEY DEFAULT
        gen_random_uuid(),
    type              VARCHAR(30) NOT NULL,
    description        TEXT NOT NULL,
    rationale          TEXT NOT NULL,
    proposed_action    JSONB NOT NULL,
    confidence         FLOAT NOT NULL,
    risk_level         VARCHAR(10) NOT NULL,
    auto_approve       BOOLEAN NOT NULL DEFAULT FALSE,
    status             VARCHAR(20) NOT NULL DEFAULT 'PENDING',
    applied_by         TEXT,
    applied_at         TIMESTAMPTZ,
    result             JSONB,
    created_at         TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at         TIMESTAMPTZ NOT NULL DEFAULT NOW(),

    CONSTRAINT la_confidence_range CHECK (confidence >= 0.0
        AND confidence <= 1.0),
    CONSTRAINT la_risk_valid        CHECK (risk_level IN (
        'LOW', 'MEDIUM', 'HIGH', 'CRITICAL'
```

```
)),
CONSTRAINT la_status_valid CHECK (status IN (
    'PENDING', 'APPROVED', 'REJECTED', 'APPLIED',
    'FAILED', 'EXPIRED'
)),
CONSTRAINT la_type_valid CHECK (type IN (
    'SCALE_UP', 'SCALE_DOWN', 'MIGRATE', 'REBALANCE',
    'DRAIN', 'EVICT', 'ALERT', 'OPTIMIZE', 'SCHEDULE'
))
);
```

Column Descriptions

Column	Type	Nullable	Default	Description
id	UUID	NO	gen_random_uuid()	Primary key; advisory identifier
type	VARCHAR(30)	NO	—	Advisory category (e.g., SCALE_UP, MIGRATE, ALERT)
description	TEXT	NO	—	Human-readable description of the proposed action
rationale	TEXT	NO	—	Explanation of why the advisory was generated
proposed_action	JSONB	NO	—	Machine-readable action specification
confidence	FLOAT	NO	—	Confidence score (0.0–1.0)
risk_level	VARCHAR(10)	NO	—	Risk classification: LOW,

Column	Type	Nullable	Default	Description
				MEDIUM, HIGH, CRITICAL
auto_approve	BOOLEAN	NO	FALSE	Whether this advisory can be automatically applied without human approval
status	VARCHAR(20)	NO	' PENDING '	Approval workflow state
applied_by	TEXT	YES	NULL	Identity of who applied the advisory
applied_at	TIMESTAMPTZ	YES	NULL	When the advisory was applied
result	JSONB	YES	NULL	Outcome of applying the advisory
created_at	TIMESTAMPTZ	NO	NOW()	Advisory creation timestamp
updated_at	TIMESTAMPTZ	NO	NOW()	Last update timestamp (auto-bumped by trigger)

Index Definitions

```
CREATE INDEX IF NOT EXISTS idx_llm_advisories_status ON llm_advisories(status);
CREATE INDEX IF NOT EXISTS idx_llm_advisories_type ON llm_advisories(type);
```



```
CREATE INDEX IF NOT EXISTS idx_llm_advisories_risk_level ON
  llm_advisories(risk_level);
CREATE INDEX IF NOT EXISTS idx_llm_advisories_created_at ON
  llm_advisories(created_at);
```

Trigger Definitions

Trigger	Event	Timing	Function
helix_llm_advisories_updated_at	UPDATE	BEFORE	helix_update_updated_at

1.10 audit_log (Migration 010)

The `audit_log` table is a range-partitioned append-only log of all cluster mutations. It captures every INSERT, UPDATE, and DELETE on audited tables via the `helix_audit_trigger()` function. Monthly partitions enable efficient time-range queries and data retention management.

CREATE TABLE Statement

```
CREATE TABLE IF NOT EXISTS audit_log (
  id          BIGSERIAL,
  timestamp   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  event_type  VARCHAR(50) NOT NULL,
  severity    VARCHAR(10) NOT NULL DEFAULT 'INFO',
  actor       TEXT NOT NULL,
  resource_type VARCHAR(50) NOT NULL,
  resource_id TEXT,
  action      VARCHAR(50) NOT NULL,
  details     JSONB NOT NULL DEFAULT '{}',
  source_ip   INET,
  session_id  UUID,
  PRIMARY KEY (id, timestamp),

  CONSTRAINT audit_log_severity_valid CHECK (severity IN (
    'DEBUG', 'INFO', 'WARNING', 'ERROR', 'CRITICAL'
  ))
) PARTITION BY RANGE (timestamp);
```

Partition Definitions

```
CREATE TABLE IF NOT EXISTS audit_log_default
  PARTITION OF audit_log DEFAULT;
```

```
-- Monthly partitions for 2026
```

```

CREATE TABLE IF NOT EXISTS audit_log_2026_06
    PARTITION OF audit_log FOR VALUES FROM ('2026-06-01')
    TO ('2026-07-01');
CREATE TABLE IF NOT EXISTS audit_log_2026_07
    PARTITION OF audit_log FOR VALUES FROM ('2026-07-01')
    TO ('2026-08-01');
CREATE TABLE IF NOT EXISTS audit_log_2026_08
    PARTITION OF audit_log FOR VALUES FROM ('2026-08-01')
    TO ('2026-09-01');
CREATE TABLE IF NOT EXISTS audit_log_2026_09
    PARTITION OF audit_log FOR VALUES FROM ('2026-09-01')
    TO ('2026-10-01');
CREATE TABLE IF NOT EXISTS audit_log_2026_10
    PARTITION OF audit_log FOR VALUES FROM ('2026-10-01')
    TO ('2026-11-01');
CREATE TABLE IF NOT EXISTS audit_log_2026_11
    PARTITION OF audit_log FOR VALUES FROM ('2026-11-01')
    TO ('2026-12-01');
CREATE TABLE IF NOT EXISTS audit_log_2026_12
    PARTITION OF audit_log FOR VALUES FROM ('2026-12-01')
    TO ('2027-01-01');

-- Monthly partitions for 2027
CREATE TABLE IF NOT EXISTS audit_log_2027_01
    PARTITION OF audit_log FOR VALUES FROM ('2027-01-01')
    TO ('2027-02-01');
CREATE TABLE IF NOT EXISTS audit_log_2027_02
    PARTITION OF audit_log FOR VALUES FROM ('2027-02-01')
    TO ('2027-03-01');
CREATE TABLE IF NOT EXISTS audit_log_2027_03
    PARTITION OF audit_log FOR VALUES FROM ('2027-03-01')
    TO ('2027-04-01');
CREATE TABLE IF NOT EXISTS audit_log_2027_04
    PARTITION OF audit_log FOR VALUES FROM ('2027-04-01')
    TO ('2027-05-01');
CREATE TABLE IF NOT EXISTS audit_log_2027_05
    PARTITION OF audit_log FOR VALUES FROM ('2027-05-01')
    TO ('2027-06-01');
CREATE TABLE IF NOT EXISTS audit_log_2027_06
    PARTITION OF audit_log FOR VALUES FROM ('2027-06-01')
    TO ('2027-07-01');

```

Column Descriptions

Column	Type	Nullable	Default	Description
id	BIGSERIAL	NO	auto	Auto-incrementing sequence (part of

Column	Type	Nullable	Default	Description
				composite PK with timestamp)
timestamp	TIMESTAMPTZ	NO	NOW()	Event timestamp (partition key)
event_type	VARCHAR(50)	NO	—	Event type: INSERT nodes , UPDATE sessions , DELETE_users , etc.
severity	VARCHAR(10)	NO	'INFO'	Event severity: DEBUG, INFO, WARNING, ERROR, CRITICAL
actor	TEXT	NO	—	Identity that performed the action (from app.current_user session variable)
resource_type	TEXT	NO	—	Table name of the affected resource
resource_id	TEXT	YES	NULL	Primary key of the affected row
action	VARCHAR(50)	NO	—	Operation: INSERT, UPDATE, DELETE
details	JSONB	NO	'{}'	Full row snapshot (OLD for DELETE, NEW for INSERT/UPDATE)
source_ip	INET	YES	NULL	Source IP address of the request
session_id	UUID	YES	NULL	Associated session ID if applicable

Index Definitions

```

CREATE INDEX IF NOT EXISTS idx_audit_log_timestamp ON
  audit_log(timestamp);
CREATE INDEX IF NOT EXISTS idx_audit_log_event_type ON
  audit_log(event_type);

```

```

CREATE INDEX IF NOT EXISTS idx_audit_log_actor      ON
    audit_log(actor);
CREATE INDEX IF NOT EXISTS idx_audit_log_resource  ON
    audit_log(resource_type, resource_id);
CREATE INDEX IF NOT EXISTS idx_audit_log_severity  ON
    audit_log(severity);
CREATE INDEX IF NOT EXISTS idx_audit_log_session_id ON
    audit_log(session_id);

```

1.11 build_jobs (Migration 011)

The `build_jobs` table tracks build and compilation jobs submitted to the cluster's build service. Each job has resource requirements, priority, and a lifecycle from PENDING through completion.

CREATE TABLE Statement

```

CREATE TABLE IF NOT EXISTS build_jobs (
    id                UUID PRIMARY KEY DEFAULT
        gen_random_uuid(),
    name              VARCHAR(255) NOT NULL,
    status            VARCHAR(20) NOT NULL DEFAULT 'PENDING',
    mode              VARCHAR(20) NOT NULL DEFAULT 'BATCH',
    cpu_request       INT NOT NULL DEFAULT 1000,
    memory_request    BIGINT NOT NULL DEFAULT 1073741824,
    gpu_request       JSONB DEFAULT NULL,
    node_id           UUID REFERENCES nodes(id),
    priority          INT NOT NULL DEFAULT 50,
    labels            JSONB NOT NULL DEFAULT '{}',
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    started_at        TIMESTAMPTZ,
    completed_at      TIMESTAMPTZ,
    updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),

    CONSTRAINT bj_status_valid CHECK (status IN (
        'PENDING', 'QUEUED', 'RUNNING', 'SUCCEEDED',
        'FAILED', 'CANCELLED'
    )),
    CONSTRAINT bj_mode_valid CHECK (mode IN (
        'BATCH', 'STREAMING', 'INCREMENTAL', 'PARALLEL'
    )),
    CONSTRAINT bj_priority_range CHECK (priority >= 0 AND
        priority <= 100),
    CONSTRAINT bj_cpu_positive CHECK (cpu_request > 0),

```

```
);  
    CONSTRAINT bj_mem_positive    CHECK (memory_request > 0)
```

Index Definitions

```
CREATE INDEX IF NOT EXISTS idx_build_jobs_status    ON  
    build_jobs(status);  
CREATE INDEX IF NOT EXISTS idx_build_jobs_node_id    ON  
    build_jobs(node_id);  
CREATE INDEX IF NOT EXISTS idx_build_jobs_mode    ON  
    build_jobs(mode);  
CREATE INDEX IF NOT EXISTS idx_build_jobs_created_at    ON  
    build_jobs(created_at);  
CREATE INDEX IF NOT EXISTS idx_build_jobs_priority    ON  
    build_jobs(priority);
```

Foreign Key Relationships

- build_jobs.node_id → nodes.id
- Referenced by: build_artifacts.job_id

Trigger Definitions

Trigger	Event	Timing	Function
helix_build_jobs_updated_at	UPDATE	BEFORE	helix_update_updated_at_
helix_build_jobs_audit	INSERT, UPDATE, DELETE	AFTER	helix_audit_trigger()

1.12 build_artifacts (Migration 012)

The `build_artifacts` table stores metadata for files produced by build jobs, including content hashes, sizes, storage paths, and type classification.

CREATE TABLE Statement

```
CREATE TABLE IF NOT EXISTS build_artifacts (  
    id                UUID PRIMARY KEY DEFAULT  
        gen_random_uuid(),  
    job_id            UUID NOT NULL REFERENCES build_jobs(id)  
        ON DELETE CASCADE,  
    artifact_hash     TEXT NOT NULL,
```

```

size_bytes      BIGINT NOT NULL,
storage_path    TEXT NOT NULL,
artifact_type   VARCHAR(30) NOT NULL DEFAULT 'BINARY',
metadata        JSONB NOT NULL DEFAULT '{}',
created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),

CONSTRAINT ba_size_positive    CHECK (size_bytes > 0),
CONSTRAINT ba_type_valid      CHECK (artifact_type IN (
    'BINARY', 'CONTAINER_IMAGE', 'WASM', 'LIBRARY',
    'CONFIG', 'REPORT'
))
);

```

Index Definitions

```

CREATE INDEX IF NOT EXISTS idx_build_artifacts_job_id
    ON build_artifacts(job_id);
CREATE INDEX IF NOT EXISTS idx_build_artifacts_hash
    ON build_artifacts(artifact_hash);
CREATE INDEX IF NOT EXISTS idx_build_artifacts_created_at
    ON build_artifacts(created_at);
CREATE INDEX IF NOT EXISTS idx_build_artifacts_type
    ON build_artifacts(artifact_type);

```

Foreign Key Relationships

- `build_artifacts.job_id` → `build_jobs.id` (ON DELETE CASCADE)

1.13 users (Migration 013)

The `users` table is a shadow of the OIDC identity provider. It stores role assignments, resource quotas, and labels for each authenticated user, keyed by their SPIFFE identity.

CREATE TABLE Statement

```

CREATE TABLE IF NOT EXISTS users (
    id          UUID PRIMARY KEY DEFAULT
        gen_random_uuid(),
    spiffe_id    TEXT NOT NULL,
    email        TEXT,
    name         VARCHAR(255),
    role         VARCHAR(20) NOT NULL DEFAULT 'USER',
    quota_cpu    INT NOT NULL DEFAULT 8000,

```

```

quota_memory    BIGINT NOT NULL DEFAULT 17179869184,
quota_gpu       INT NOT NULL DEFAULT 0,
labels          JSONB NOT NULL DEFAULT '{}',
last_login      TIMESTAMPTZ,
created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),

CONSTRAINT users_spiffe_id_unique UNIQUE (spiffe_id),
CONSTRAINT users_role_valid CHECK (role IN (
    'SUPERADMIN', 'ADMIN', 'OPERATOR', 'USER',
    'READONLY', 'SERVICE'
)),
CONSTRAINT users_quota_cpu_nonneg CHECK (quota_cpu >=
    0),
CONSTRAINT users_quota_mem_nonneg CHECK (quota_memory
    >= 0),
CONSTRAINT users_quota_gpu_nonneg CHECK (quota_gpu >=
    0)
);

```

Column Descriptions

Column	Type	Nullable	Default	Description
id	UUID	NO	gen_random_uuid()	Primary key
spiffe_id	TEXT	NO	—	SPIFFE identity URI; UNIQUE; the identity backbone key
email	TEXT	YES	NULL	User email (from OIDC provider)
name	VARCHAR(255)	YES	NULL	Display name
role	VARCHAR(20)	NO	'USER'	RBAC role: SUPERADMIN, ADMIN, OPERATOR, USER, READONLY, SERVICE
quota_cpu	INT	NO	8000	CPU quota in millicores

Column	Type	Nullable	Default	Description
				(default 8 cores)
quota_memory	BIGINT	NO	17179869184	Memory quota in bytes (default 16 GiB)
quota_gpu	INT	NO	0	GPU quota count
labels	JSONB	NO	'{}'	User labels for policy evaluation
last_login	TIMESTAMPTZ	YES	NULL	Last successful authentication timestamp
created_at	TIMESTAMPTZ	NO	NOW()	User record creation timestamp
updated_at	TIMESTAMPTZ	NO	NOW()	Last update timestamp (auto-bumped by trigger)

Index Definitions

```
CREATE INDEX IF NOT EXISTS idx_users_spiffe_id ON
  users(spiffe_id);
CREATE INDEX IF NOT EXISTS idx_users_role ON
  users(role);
CREATE INDEX IF NOT EXISTS idx_users_email ON
  users(email);
CREATE INDEX IF NOT EXISTS idx_users_labels ON users
  USING GIN(labels);
```

Trigger Definitions

Trigger	Event	Timing	Function
helix_users_updated_at	UPDATE	BEFORE	helix_update_updated_at_column
helix_users_audit		AFTER	helix_audit_trigger()

Trigger	Event	Timing	Function
	INSERT, UPDATE, DELETE		

1.14 migration_history (Migration 014)

The `migration_history` table records session migration events — when a live session is moved from one node to another for load balancing, maintenance, or failure recovery.

CREATE TABLE Statement

```
CREATE TABLE IF NOT EXISTS migration_history (
  id          UUID PRIMARY KEY DEFAULT
             gen_random_uuid(),
  session_id  UUID NOT NULL REFERENCES sessions(id),
  source_node UUID NOT NULL REFERENCES nodes(id),
  target_node UUID NOT NULL REFERENCES nodes(id),
  method      VARCHAR(20) NOT NULL,
  duration_ms INT NOT NULL,
  data_size_bytes BIGINT,
  success     BOOLEAN NOT NULL,
  error_message TEXT,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),

  CONSTRAINT mh_method_valid CHECK (method IN (
    'CHECKPOINT', 'LIVE', 'COLD', 'SNAPSHOT'
  )),
  CONSTRAINT mh_duration_nonneg CHECK (duration_ms >=
    0),
  CONSTRAINT mh_nodes_differ CHECK (source_node <>
    target_node)
);
```

Column Descriptions

Column	Type	Nullable	Default	Description
<code>id</code>	UUID	NO	<code>gen_random_uuid()</code>	Primary key
<code>session_id</code>	UUID	NO	—	Session that was migrated

Column	Type	Nullable	Default	Description
source_node	UUID	NO	—	Node the session migrated FROM
target_node	UUID	NO	—	Node the session migrated TO
method	VARCHAR(20)	NO	—	Migration technique: CHECKPOINT (CRIU), LIVE, COLD, SNAPSHOT
duration_ms	INT	NO	—	Total migration duration in milliseconds
data_size_bytes	BIGINT	YES	NULL	Amount of state transferred
success	BOOLEAN	NO	—	Whether the migration succeeded
error_message	TEXT	YES	NULL	Error description if migration failed
created_at	TIMESTAMPTZ	NO	NOW()	Migration event timestamp

Index Definitions

```

CREATE INDEX IF NOT EXISTS
    idx_migration_history_session_id ON
    migration_history(session_id);
CREATE INDEX IF NOT EXISTS
    idx_migration_history_source_node ON
    migration_history(source_node);

```

```
CREATE INDEX IF NOT EXISTS
    idx_migration_history_target_node ON
    migration_history(target_node);
CREATE INDEX IF NOT EXISTS
    idx_migration_history_success ON
    migration_history(success);
```

Constraint Notes

- `mh_nodes_differ` ensures source and target are different (prevents no-op migrations)
- Note: there is no ON DELETE CASCADE on the foreign keys for `sessions(id)` and `nodes(id)`, meaning migration history is preserved even if the source session or nodes are deleted.

1.15 Triggers and Functions (Migration 015)

Migration 015 also introduces two additional tables: `network_policies` and `cluster_config`, plus the trigger functions and seed data.

Function: `helix_update_updated_at_column()`

```
CREATE OR REPLACE FUNCTION helix_update_updated_at_column()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

This function is invoked by BEFORE UPDATE triggers on all tables with an `updated_at` column. It ensures that `updated_at` is always set to the current transaction timestamp, preventing applications from accidentally overwriting it.

Function: `helix_audit_trigger()`

```
CREATE OR REPLACE FUNCTION helix_audit_trigger()
RETURNS TRIGGER AS $$
BEGIN
    IF (TG_OP = 'DELETE') THEN
        INSERT INTO audit_log (
            event_type, severity, actor,
            resource_type, resource_id, action, details
        ) VALUES (
            TG_OP || '_' || TG_TABLE_NAME,
```

```

        'WARNING',
        COALESCE(current_setting('app.current_user',
TRUE), 'system'),
        TG_TABLE_NAME,
        OLD.id::TEXT,
        TG_OP,
        to_jsonb(OLD)
    );
    RETURN OLD;
ELSE
    INSERT INTO audit_log (
        event_type, severity, actor,
        resource_type, resource_id, action, details
    ) VALUES (
        TG_OP || '_' || TG_TABLE_NAME,
        'INFO',
        COALESCE(current_setting('app.current_user',
TRUE), 'system'),
        TG_TABLE_NAME,
        NEW.id::TEXT,
        TG_OP,
        to_jsonb(NEW)
    );
    RETURN NEW;
END IF;
END;
$$ LANGUAGE plpgsql;

```

Security note: This function is `SECURITY DEFINER` to ensure it always runs with sufficient privilege to `INSERT` into `audit_log`. It captures:

- **DELETE** operations at severity `WARNING` with the OLD row state
- **INSERT/UPDATE** operations at severity `INFO` with the NEW row state
- The actor is resolved from the PostgreSQL session variable `app.current_user`
- Full row state is captured as JSONB in the `details` column

Complete Trigger Map

Trigger	Table	Event	Timing	Fu
<code>helix_nodes_updated_at</code>	<code>nodes</code>	UPDATE	BEFORE	h
<code>helix_gpu_devices_updated_at</code>	<code>gpu_devices</code>	UPDATE	BEFORE	h
<code>helix_sessions_updated_at</code>	<code>sessions</code>	UPDATE	BEFORE	h
<code>helix_reservations_updated_at</code>	<code>reservations</code>	UPDATE	BEFORE	h

Trigger	Table	Event	Timing	Function
helix_scheduling_queue_updated_at	scheduling_queue	UPDATE	BEFORE	helix_scheduling_queue_updated_at
helix_llm_advisories_updated_at	llm_advisories	UPDATE	BEFORE	helix_llm_advisories_updated_at
helix_build_jobs_updated_at	build_jobs	UPDATE	BEFORE	helix_build_jobs_updated_at
helix_users_updated_at	users	UPDATE	BEFORE	helix_users_updated_at
helix_network_policies_updated_at	network_policies	UPDATE	BEFORE	helix_network_policies_updated_at
helix_cluster_config_updated_at	cluster_config	UPDATE	BEFORE	helix_cluster_config_updated_at
helix_nodes_audit	nodes	INSERT, UPDATE, DELETE	AFTER	helix_nodes_audit
helix_sessions_audit	sessions	INSERT, UPDATE, DELETE	AFTER	helix_sessions_audit
helix_build_jobs_audit	build_jobs	INSERT, UPDATE, DELETE	AFTER	helix_build_jobs_audit
helix_users_audit	users	INSERT, UPDATE, DELETE	AFTER	helix_users_audit

network_policies (Migration 015)

```

CREATE TABLE IF NOT EXISTS network_policies (
  id          UUID PRIMARY KEY DEFAULT
              gen_random_uuid(),
  name        VARCHAR(255) NOT NULL,
  direction   VARCHAR(10) NOT NULL,
  action       VARCHAR(10) NOT NULL,
  protocol     VARCHAR(10),
  src_cidr     CIDR,
  dst_cidr     CIDR,
  src_port_min INT,
  src_port_max INT,
  dst_port_min INT,
  dst_port_max INT,
  priority     INT NOT NULL DEFAULT 100,
  enabled      BOOLEAN NOT NULL DEFAULT TRUE,
  labels       JSONB NOT NULL DEFAULT '{}',
  description   TEXT,
  created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),

```

```

updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),

CONSTRAINT np_name_unique          UNIQUE (name),
CONSTRAINT np_direction_valid      CHECK (direction IN
    ('INGRESS', 'EGRESS', 'BOTH')),
CONSTRAINT np_action_valid         CHECK (action IN
    ('ALLOW', 'DENY', 'LOG', 'RATE_LIMIT')),
CONSTRAINT np_protocol_valid       CHECK (protocol IS
    NULL OR protocol IN (
    'TCP', 'UDP', 'ICMP', 'ESP', 'AH', 'ANY'
    )),
CONSTRAINT np_priority_range       CHECK (priority >= 0
    AND priority <= 65535),
CONSTRAINT np_src_port_range       CHECK (
    src_port_min IS NULL OR (src_port_min >= 0 AND
    src_port_min <= 65535)
    ),
CONSTRAINT np_dst_port_range       CHECK (
    dst_port_min IS NULL OR (dst_port_min >= 0 AND
    dst_port_min <= 65535)
    ),
CONSTRAINT np_src_port_order       CHECK (
    src_port_min IS NULL OR src_port_max IS NULL OR
    src_port_min <= src_port_max
    ),
CONSTRAINT np_dst_port_order       CHECK (
    dst_port_min IS NULL OR dst_port_max IS NULL OR
    dst_port_min <= dst_port_max
    )
);

```

cluster_config (Migration 015)

```

CREATE TABLE IF NOT EXISTS cluster_config (
    id          UUID PRIMARY KEY DEFAULT
        gen_random_uuid(),
    key         VARCHAR(255) NOT NULL,
    value       JSONB NOT NULL,
    description  TEXT,
    category    VARCHAR(50) NOT NULL DEFAULT 'general',
    readonly    BOOLEAN NOT NULL DEFAULT FALSE,
    version     INT NOT NULL DEFAULT 1,
    updated_by  TEXT NOT NULL DEFAULT 'system',
    created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),

    CONSTRAINT cc_key_unique          UNIQUE (key),

```

```

CONSTRAINT cc_version_positive CHECK (version > 0),
CONSTRAINT cc_category_valid CHECK (category IN (
    'general', 'scheduler', 'network', 'security',
    'storage', 'monitoring', 'llm', 'build'
))
);

```

Seed Data

```

INSERT INTO cluster_config (key, value, description,
    category) VALUES
    ('scheduler.max_queue_depth', '1000', 'Maximum
    scheduler queue depth', 'scheduler'),
    ('scheduler.default_priority', '50', 'Default
    session scheduling priority', 'scheduler'),
    ('network.mtu', '1420', 'WireGuard
    MTU (bytes)', 'network'),
    ('security.session_ttl_hours', '8', 'Default
    session TTL in hours', 'security'),
    ('monitoring.health_interval_s', '30', 'Health
    snapshot interval (seconds)', 'monitoring')
ON CONFLICT (key) DO NOTHING;

```

1.16 Complete Schema Summary

Entity-Relationship Diagram (Text)

nodes

	1:N	gpu devices (node id, CASCADE)
	1:N	sessions (node id, SET NULL)
	1:N	session panes (node id, SET NULL)
	1:N	reservations (node id, RESTRICT)
	1:N	health snapshots (node id, CASCADE)
	1:N	build jobs (node id, SET NULL)
	N:M	migration_history (source_node, target_node)

sessions

	1:N	session windows (session id, CASCADE)
	1:N	reservations (session id, CASCADE)
	1:N	migration_history (session_id, RESTRICT)

session windows

	1:N	session_panes (window_id, CASCADE)
--	-----	------------------------------------

gpu devices

| 1:N session_panes (gpu_id, SET NULL)

build jobs

| 1:N build_artifacts (job_id, CASCADE)

Table Count

#	Table	Primary Key	Partitioned?	Audited?
1	nodes	UUID	No	Yes
2	gpu_devices	UUID	No	No (updated_at only)
3	sessions	UUID	No	Yes
4	session_windows	UUID	No	No
5	session_panes	UUID	No	No
6	reservations	UUID	No	No (updated_at only)
7	scheduling_queue	UUID	No	No (updated_at only)
8	health_snapshots	BIGSERIAL	No	No
9	llm_advisories	UUID	No	No (updated_at only)
10	audit_log	(BIGSERIAL, TIMESTAMPTZ)	Yes (RANGE)	N/A
11	build_jobs	UUID	No	Yes
12	build_artifacts	UUID	No	No
13	users	UUID	No	Yes
14	migration_history	UUID	No	No
15	network_policies	UUID	No	

#	Table	Primary Key	Partitioned?	Audited?
				No (updated_at only)
16	cluster_config	UUID	No	No (updated_at only)

Total Index Count: 53

Part 2: DQLite Schema

DQLite provides Raft-replicated SQLite for per-node state that must survive node restarts and be consistent across local replicas. The schema is defined in `migrations/dqlite/001_node_local_schema.sql`.

2.1 Complete DQLite Schema

```
-- Local node identity and configuration
CREATE TABLE IF NOT EXISTS local_node (
    node_id          TEXT PRIMARY KEY,
    hostname         TEXT NOT NULL,
    wg_pubkey       TEXT NOT NULL UNIQUE,
    spiffe_id       TEXT NOT NULL UNIQUE,
    role            TEXT NOT NULL DEFAULT 'WORKER',
    region          TEXT,
    version         TEXT NOT NULL,
    config          BLOB,
    joined_at       DATETIME NOT NULL DEFAULT
        CURRENT_TIMESTAMP,
    updated_at      DATETIME NOT NULL DEFAULT
        CURRENT_TIMESTAMP
);

-- Local resource inventory (snapshot, refreshed on change)
CREATE TABLE IF NOT EXISTS local_resources (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    resource_type   TEXT NOT NULL,
    name           TEXT NOT NULL,
    total          INTEGER NOT NULL,
    available      INTEGER NOT NULL,
    unit           TEXT NOT NULL,
    attributes     BLOB,
```

```

        updated_at      DATETIME NOT NULL DEFAULT
                        CURRENT_TIMESTAMP
    );

-- Local GPU devices (subset of PostgreSQL gpu_devices)
CREATE TABLE IF NOT EXISTS local_gpus (
    id                  TEXT PRIMARY KEY,
    vendor              TEXT NOT NULL,
    model              TEXT NOT NULL,
    driver_version      TEXT NOT NULL,
    api                 TEXT NOT NULL,
    total_memory        INTEGER NOT NULL,
    compute_units       INTEGER NOT NULL,
    status              TEXT NOT NULL DEFAULT 'AVAILABLE',
    allocated_to        TEXT,
    updated_at          DATETIME NOT NULL DEFAULT
                        CURRENT_TIMESTAMP
);

-- Active local sessions and their backends
CREATE TABLE IF NOT EXISTS local_sessions (
    session_id          TEXT PRIMARY KEY,
    name                TEXT NOT NULL,
    owner               TEXT NOT NULL,
    status              TEXT NOT NULL DEFAULT 'CREATING',
    backend              TEXT NOT NULL DEFAULT 'TMUX',
    backend_pid         INTEGER,
    node_id             TEXT,
    resources           BLOB,
    started_at          DATETIME,
    updated_at          DATETIME NOT NULL DEFAULT
                        CURRENT_TIMESTAMP
);

-- Local session windows (CRDT state stored as blob)
CREATE TABLE IF NOT EXISTS local_windows (
    window_id           TEXT PRIMARY KEY,
    session_id          TEXT NOT NULL REFERENCES
        local_sessions(session_id) ON DELETE CASCADE,
    name                TEXT NOT NULL,
    layout              TEXT NOT NULL DEFAULT 'tiled',
    active              INTEGER NOT NULL DEFAULT 0,
    crdt_state          BLOB,
    created_at          DATETIME NOT NULL DEFAULT
                        CURRENT_TIMESTAMP
);

-- Local session panes

```

```

CREATE TABLE IF NOT EXISTS local_panes (
    pane_id          TEXT PRIMARY KEY,
    window_id        TEXT NOT NULL REFERENCES
        local_windows(window_id) ON DELETE CASCADE,
    command          TEXT,
    working_dir       TEXT,
    environment       BLOB,
    pty_path          TEXT,
    status            TEXT NOT NULL DEFAULT 'CREATING',
    crdt_state        BLOB,
    created_at        DATETIME NOT NULL DEFAULT
        CURRENT_TIMESTAMP
);

```

-- Peer WireGuard endpoints (for mesh reconnection)

```

CREATE TABLE IF NOT EXISTS local_peers (
    node_id          TEXT PRIMARY KEY,
    hostname          TEXT NOT NULL,
    wg_pubkey         TEXT NOT NULL UNIQUE,
    endpoint_host     TEXT,
    endpoint_port     INTEGER,
    allowed_ips       TEXT,
    last_handshake    INTEGER,
    status            TEXT NOT NULL DEFAULT 'ACTIVE',
    updated_at        DATETIME NOT NULL DEFAULT
        CURRENT_TIMESTAMP
);

```

-- Local task queue (build jobs, scheduled tasks)

```

CREATE TABLE IF NOT EXISTS local_tasks (
    task_id          TEXT PRIMARY KEY,
    task_type         TEXT NOT NULL,
    payload           BLOB NOT NULL,
    priority          INTEGER NOT NULL DEFAULT 50,
    status            TEXT NOT NULL DEFAULT 'PENDING',
    retry_count       INTEGER NOT NULL DEFAULT 0,
    max_retries       INTEGER NOT NULL DEFAULT 3,
    scheduled_at      DATETIME,
    started_at        DATETIME,
    completed_at      DATETIME,
    error_message     TEXT,
    created_at        DATETIME NOT NULL DEFAULT
        CURRENT_TIMESTAMP,
    updated_at        DATETIME NOT NULL DEFAULT
        CURRENT_TIMESTAMP
);

```

-- Local health metrics cache (last N samples)

```
CREATE TABLE IF NOT EXISTS local_health (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  overall_score INTEGER NOT NULL,
  cpu_score   INTEGER NOT NULL,
  memory_score INTEGER NOT NULL,
  disk_score  INTEGER NOT NULL,
  network_score INTEGER NOT NULL,
  gpu_score   INTEGER NOT NULL,
  temperature_score INTEGER NOT NULL,
  services_score INTEGER NOT NULL,
  metrics      BLOB,
  recorded_at  DATETIME NOT NULL DEFAULT
    CURRENT_TIMESTAMP
);
```

2.2 DQLite Index Definitions

```
CREATE INDEX IF NOT EXISTS idx_local_sessions_status ON
  local_sessions(status);
CREATE INDEX IF NOT EXISTS idx_local_tasks_status ON
  local_tasks(status);
CREATE INDEX IF NOT EXISTS idx_local_tasks_scheduled ON
  local_tasks(scheduled_at);
CREATE INDEX IF NOT EXISTS idx_local_health_recorded ON
  local_health(recorded_at);
CREATE INDEX IF NOT EXISTS idx_local_peers_status ON
  local_peers(status);
```

2.3 DQLite vs PostgreSQL Comparison

Aspect	PostgreSQL (Central)	DQLite (Per-Node)
Scope	Cluster-wide	Single node
Consensus	Primary/Standby	Raft (dqlite)
Key type	UUID	TEXT (UUID string)
CRDT storage	JSONB	BLOB (serialized)
Rich types	INET, INET[], CIDR	TEXT approximations
Timestamps	TIMESTAMPTZ	DATETIME
Auto-increment	BIGSERIAL	INTEGER AUTOINCREMENT
Partitions	Yes (audit_log)	N/A

Aspect	PostgreSQL (Central)	DQLite (Per-Node)
Triggers	Yes (updated_at, audit)	None
Synchronization	Writes from services	Heartbeat from agent

Part 3: SQLite Registry Schema

The HXC Registry database (`data/hxc_registry.db`) is a SQLite database that serves as the single source of truth for all Helix work items (tickets). It is managed by `cmd/hxc-registry` and `pkg/hxcregistry/`.

3.1 items

```
CREATE TABLE items (
  hxc_id          TEXT PRIMARY KEY NOT NULL,
  type            TEXT NOT NULL CHECK (type IN ('Bug',
    'Feature', 'Task', 'Research', 'Docs')),
  status          TEXT NOT NULL CHECK (status IN
    ('Queued', 'In progress', 'Ready for testing', 'In
    testing', 'Completed', 'Obsolete')),
  priority        TEXT NOT NULL CHECK (priority IN
    ('P0', 'P1', 'P2', 'P3')),
  phase          INTEGER NOT NULL DEFAULT 0,
  title           TEXT NOT NULL,
  description     TEXT NOT NULL,
  commit_sha      TEXT,
  forensic_anchor TEXT,
  closure_criteria TEXT,
  composes_with   TEXT,
  current_location TEXT NOT NULL CHECK (current_location
    IN ('Issues', 'Fixed')) DEFAULT 'Issues',
  heading_hash    TEXT NOT NULL UNIQUE,
  created_at      TEXT NOT NULL DEFAULT
    (datetime('now')),
  last_modified   TEXT NOT NULL DEFAULT (datetime('now'))
);
```

Column Descriptions

Column	Type	Description
<code>hxc_id</code>	TEXT PK	Ticket identifier (e.g., "HXC-001"); permanent and never reused

Column	Type	Description
type	TEXT	Work type: Bug, Feature, Task, Research, Docs
status	TEXT	Lifecycle: Queued → In progress → Ready for testing → In testing → Completed / Obsolete
priority	TEXT	P0 (critical) through P3 (nice-to-have)
phase	INTEGER	Development phase (0-8+); see Phase Legend in HXC_REGISTRY.md
title	TEXT	Short descriptive title
description	TEXT	Full description with acceptance criteria
commit_sha	TEXT	Git commit SHA that resolved this ticket
forensic_anchor	TEXT	Evidence reference for anti-bluff verification
closure_criteria	TEXT	Conditions that must be met for ticket closure
composes_with	TEXT	Related ticket IDs that compose together
current_location	TEXT	Whether the ticket is in the 'Issues' or 'Fixed' section of the registry document
heading_hash	TEXT	Content hash of the heading for integrity verification; UNIQUE

3.2 item_history

```
CREATE TABLE item_history (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  hxc_id      TEXT NOT NULL REFERENCES items(hxc_id)
              ON DELETE CASCADE,
  event_type  TEXT NOT NULL CHECK (event_type IN
    ('Opened', 'Updated', 'StatusChanged', 'Completed',
     'Obsolete')),
```

```

    by_entity      TEXT CHECK (by_entity IN ('AI',
        'User', 'System', NULL)),
    on_date         TEXT NOT NULL DEFAULT
        (datetime('now')),
    reason          TEXT,
    evidence_path   TEXT,
    created_at      TEXT NOT NULL DEFAULT (datetime('now'))
);

```

3.3 document_sources

```

CREATE TABLE document_sources (
    location        TEXT PRIMARY KEY NOT NULL CHECK
        (location IN ('Issues', 'Fixed')),
    raw_text        TEXT NOT NULL DEFAULT '',
    sha256          TEXT NOT NULL DEFAULT '',
    last_modified   TEXT NOT NULL DEFAULT (datetime('now'))
);

```

3.4 meta

```

CREATE TABLE meta (
    key             TEXT PRIMARY KEY,
    value           TEXT NOT NULL,
    last_modified   TEXT NOT NULL DEFAULT (datetime('now'))
);

```

3.5 Registry Indexes

```

CREATE INDEX idx_items_location ON items(current_location);
CREATE INDEX idx_items_phase ON items(phase);
CREATE INDEX idx_items_priority ON items(priority);
CREATE INDEX idx_items_status ON items(status);
CREATE INDEX idx_items_type ON items(type);
CREATE INDEX idx_item_history_event_type ON
    item_history(event_type);
CREATE INDEX idx_item_history_hxc_id ON
    item_history(hxc_id);

```

Part 4: etcd Key Schema

etcd serves as the cluster-wide consistent state store, providing strong consistency (Raft consensus) for critical cluster metadata. All keys are prefixed with `/clusteros/`.

4.1 Key Hierarchy

```
/clusteros/
├── nodes/
│   ├── {node id}                → Node (JSON)
│   ├── {node id}/status          → NodeStatus (ACTIVE,
│   │                               SUSPECT, LEFT, FAILED)
│   ├── {node_id}/heartbeat      → Timestamp (lease-bound, 10s
│   │                               TTL)
│   └── {node id}/leases/
│       └── {lease_id}            → Resource leases (session-
│   │                               scoped)
│   └── {node_id}/metadata        → Static node metadata
├── sessions/
│   ├── {session id}              → Session (JSON)
│   ├── {session id}/status       → SessionStatus (CREATING,
│   │                               RUNNING, MIGRATING, TERMINATED)
│   ├── {session_id}/routing      → I/O routing table (node →
│   │                               PTY mapping)
│   ├── {session_id}/bindings     → Node bindings for this
│   │                               session
│   ├── {session id}/windows      → Window list (ordered)
│   └── {session_id}/panes        → Pane list (ordered)
├── scheduler/
│   └── pool/
│       └── total                  → Aggregated cluster
resources
│   ├── available                 → Currently free resources
│   ├── by_node/{node_id}         → Per-node resource slice
│   ├── queue/
│   │   ├── {request id}          → SchedulingRequest (JSON)
│   ├── reservations/
│   │   ├── {reservation_id}      → Reservation (JSON)
│   ├── bindings/
│   │   ├── {session_id}          → NodeBinding (JSON)
├── security/
│   ├── spiffe ids/
│   │   ├── {spiffe_id_hash}      → Node ID
│   ├── wireguard/
│   │   ├── peers/
│   │   │   ├── {node_id}         → WireGuardPeerConfig (JSON)
│   │   └── subnets/
│   │       └── {subnet_cidr}      → Assignment metadata
```


└─ acl/ └─ {policy_id}	→ OPA/Rego policy bundle
reference	
└─ config/ └─ cluster/ └─ name └─ version └─ settings └─ scheduler/ └─ policy └─ weights └─ limits/ └─ {user id} └─ default └─ builds/ └─ {job id} └─ {job id}/status └─ {job_id}/artifacts └─ locks/ └─ scheduler/ └─ migrations/ └─ config/ └─ builds/ └─ leader/ └─ scheduler └─ health_monitor └─ build_coordinator └─ version/ └─ schema	→ Cluster name → Cluster version → Generic config JSON → Scheduling policy JSON → Resource weighting → UserQuota (JSON) → Default quota → BuildJob (JSON) → BuildStatus → Artifact list → Scheduling mutex (lease-bound) → Migration mutex (lease-bound) → Config changes mutex → Build orchestration mutex → Current scheduler leader → Current health monitor → Current build coordinator → Current schema version

4.2 Lease Usage

Key Pattern	Lease TTL	Purpose
/clusteros/nodes/{id}/heartbeat	10s	Node liveness; auto-expires if agent crashes
/clusteros/locks/*	30s	Distributed locks; auto-release on holder failure
/clusteros/leader/*	15s	Service leadership election; auto-

Key Pattern	Lease TTL	Purpose
		demotion on crash
/clusteros/nodes/{id}/leases/*	Session lifetime	Resource lease binding; tied to session duration

4.3 Watch Patterns

Watcher	Prefix	Purpose
Node Agent	/clusteros/nodes/{self_id}/	Receive commands targeting this node
Session Manager	/clusteros/sessions/	Track all session state changes
Scheduler	/clusteros/scheduler/queue/	Detect new scheduling requests
Health Monitor	/clusteros/nodes/{id}/heartbeat	Node failure detection
API Gateway	/clusteros/config/cluster/	Hot-reload configuration changes
Build Coordinator	/clusteros/builds/	Build job lifecycle events

4.4 Concurrency Control

1. **Optimistic locking:** All updates include the previous `mod_revision` for compare-and-swap.
2. **Transactions:** Multi-key updates use etcd transactions for atomicity.
3. **Leases:** Ephemeral keys automatically expire if the holder fails.
4. **Election:** Leaders use `clientv3/concurrency` for fair leader election.

4.5 Data Size Guidelines

- Individual values must remain < 1 MB (etcd default limit).
 - Large lists are paginated via range queries.
 - Binary data (CRDT states, large JSON) is stored in PostgreSQL; etcd holds references only.
-

Part 5: Redis Key Schema

Redis Cluster serves as the distributed cache and real-time state store. All keys are prefixed with `clusteros:`. Redis is eventually consistent; authoritative state lives in PostgreSQL and etcd.

5.1 Session State (CRDT-synced, short TTL)

Key Pattern	Type	TTL	Description
<code>clusteros:session:{id}:state</code>	Hash / JSON	60s	Session state with vector clock
<code>clusteros:session:{id}:routing</code>	Hash	60s	Node routing table for I/O
<code>clusteros:session:{id}:windows</code>	List	60s	Ordered window list
<code>clusteros:session:{id}:panes</code>	List	60s	Ordered pane list
<code>clusteros:session:{id}:active_window</code>	String	60s	Currently focused window UUID
<code>clusteros:session:{id}:crdt_clock</code>	String	60s	Vector clock JSON

5.2 Node Hot Data

Key Pattern	Type	TTL	Description
<code>clusteros:node:{id}:resources</code>	Hash	30s	Current resource availability
<code>clusteros:node:{id}:health</code>	Hash	30s	Latest health snapshot
<code>clusteros:node:{id}:metrics</code>	Sorted Set	300s	Last 5 minutes of metrics (timestamp → value)
<code>clusteros:node:{id}:heartbeat</code>	String	15s	Last heartbeat timestamp
<code>clusteros:node:{id}:capabilities</code>	Set	300s	Capability strings

5.3 GPU Status

Key Pattern	Type	TTL	Description
<code>clusteros:gpu:{id}:status</code>	String	30s	AVAILABLE , ALLOCATED , UNHEALTHY
<code>clusteros:gpu:{id}:metrics</code>	Hash	30s	Temperature, utilization, memory usage
<code>clusteros:gpu:{id}:allocated_to</code>	String	60s	Session ID if allocated

5.4 Cache Aggregates

Key Pattern	Type	TTL	Description
<code>clusteros:cache:sessions</code>	Sorted Set	60s	Session list sorted by last activity
<code>clusteros:cache:pool</code>	Hash	30s	Aggregated resource pool snapshot
<code>clusteros:cache:capabilities</code>	Set	300s	All cluster capabilities
<code>clusteros:cache:nodes:active</code>	Set	30s	Set of active node IDs
<code>clusteros:cache:nodes:by_region:{region}</code>	Set	60s	Node IDs per region

5.5 Rate Limiting

Key Pattern	Type	TTL	Description
<code>clusteros:ratelimit:{user_id}</code>	Hash	60s	Token bucket counters
<code>clusteros:ratelimit:global</code>	Hash	60s	Global rate limit state
<code>clusteros:ratelimit:burst:{resource}</code>	String	60s	Burst allowance tracker

Rate limiters use Redis Lua scripts for atomic token bucket operations.

5.6 Pub/Sub Channels

Channel	Description
<code>clusteros:events:nodes</code>	Node join / leave / fail events
<code>clusteros:events:sessions</code>	Session create / terminate / migrate
<code>clusteros:events:scheduler</code>	Scheduling decisions and queue changes
<code>clusteros:events:alerts</code>	Health alerts and predictions
<code>clusteros:events:builds</code>	Build job state changes
<code>clusteros:events:advisories</code>	LLM advisory generation

5.7 Scheduler State

Key Pattern	Type	TTL	Description
<code>clusteros:scheduler:queue</code>	Sorted Set	0	Pending scheduling requests (score = priority)
<code>clusteros:scheduler:bindings</code>	Hash	60s	Session → Node bindings
<code>clusteros:scheduler:reservations</code>	Hash	60s	Active reservation IDs

5.8 Build System

Key Pattern	Type	TTL	Description
<code>clusteros:build:{id}:status</code>	String	300s	Current build status
<code>clusteros:build:{id}:progress</code>	Hash	300s	Progress metrics
<code>clusteros:build:{id}:artifacts</code>	List	0	Artifact IDs produced

Key Pattern	Type	TTL	Description
<code>clusteros:build:queue</code>	Sorted Set	0	Pending build jobs

5.9 TTL Strategy Summary

Data Category	TTL	Rationale
Heartbeat data	15s	Fast failure detection; stale data is dangerous
Resource snapshots	30s	Frequently refreshed; stale beyond 30s
Session state	60s	CRDT sync interval; refreshed on every sync
Metrics history	300s	5-minute sliding window for dashboards
Persistent aggregates	No TTL	Refreshed by writers; never auto-expire

5.10 Consistency Notes

1. Redis is **eventually consistent** — authoritative state lives in PostgreSQL and etcd.
2. Session CRDT state uses vector clocks for conflict resolution.
3. Node heartbeats are written with `NX` (only if not exists) to detect stale entries.
4. Rate limiters use Redis Lua scripts for atomic token bucket operations.

Part 6: Schema Drift Analysis

6.1 HXC-1639: Drift Guard Mechanism

The drift guard is a build-time test (`internal/schema/drift_guard_test.go`) that ensures the consolidated `0001_primary_schema.sql` artifact is always byte-for-byte identical (after normalization) to the concatenation of the migration chain (`001_*.up.sql` through `015_*.up.sql`).

How It Works

1. `schema.ReadSQL()` reads `0001_primary_schema.sql` (the consolidated artifact).
2. `schema.ReadChainSQL()` reads and concatenates all `*_up.sql` chain files in version order.
3. Both are normalized: SQL line comments stripped, whitespace collapsed, lowercased.
4. The test `TestDriftGuard_ConsolidatedEqualsChain` asserts `normC == normChain`.
5. Additional tests assert identical table sets, index sets, trigger sets, function sets, and CHECK constraint sets.

Regeneration

If any chain file is modified, the consolidated artifact must be regenerated:

```
python3 migrations/postgresql/.gen_schema.py
```

Drift Guard Tests

Test	What It Checks
<code>TestDriftGuard_ConsolidatedEqualsChain</code>	Normalized SQL text is identical
<code>TestDriftGuard_SameTableSet</code>	Same set of CREATE TABLE names
<code>TestDriftGuard_SameIndexSet</code>	Same set of CREATE INDEX names
<code>TestDriftGuard_SameTriggerSet</code>	Same set of CREATE TRIGGER names
<code>TestDriftGuard_SameFunctionSet</code>	Same set of CREATE OR REPLACE FUNCTION names
<code>TestDriftGuard_SameCheckConstraintSet</code>	Same set of CONSTRAINT ... CHECK names
<code>TestDriftGuard_MutationDetectsDivergence</code>	A canary extra table IS detected (mutation proof)

Structural Verification

The `schema.VerifyStructure()` function performs runtime structural checks against the SQL text:

- All 15 required tables present
- Both trigger functions present
- All 10 `updated_at` triggers and 4 audit triggers present
- 23 representative index names present
- 6 CHECK constraint fragments present
- `audit_log` is `PARTITION BY RANGE`
- At least one `PARTITION OF audit_log` child exists
- `helix_nodes_audit` trigger definition present

6.2 Known Divergence Points

As of the current schema, **there is zero drift** between the consolidated artifact and the chain. The drift guard test passes.

However, there are known risk areas for future divergence:

1. **New migrations:** Adding a `016_*.up.sql` without regenerating `0001_primary_schema.sql` will immediately cause drift.
2. **Seed data in seed-data.sql:** The `scripts/seed-data.sql` references tables like `resource_allocations`, `health_scores`, and `advisories` that do NOT exist in the schema. These appear to be stale references from an earlier schema version. This is a bug that should be tracked.
3. **Session status values:** The seed data uses `'ACTIVE'` for node status, but the schema CHECK constraint only allows `JOINING`, `READY`, `DRAINING`, `OFFLINE`, `MAINTENANCE`, `EVICTED`. This would cause INSERT failures.
4. **GPU status values:** The seed data uses `'ALLOCATED'` for GPU status, but the schema CHECK constraint uses `'IN_USE'` (not `'ALLOCATED'`).
5. **Migration method values:** The seed data uses `'CRIU'`, `'RESTART'`, `'DMTCP'` for migration methods, but the schema CHECK constraint only allows `'CHECKPOINT'`, `'LIVE'`, `'COLD'`, `'SNAPSHOT'`.

6.3 HXC-1639 Remediation Plan

Phase 1: Immediate (Completed)



Drift guard test implemented and passing



Structural verification function `VerifyStructure()` implemented



`0001_primary_schema.sql` is verified identical to chain concatenation

Phase 2: Seed Data Alignment

- ☐ Fix `scripts/seed-data.sql` to use correct table names (`reservations` not `resource allocations` , `health snapshots` not `health_scores` , `llm_advisories` not `advisories`)
- ☐ Fix node status values in seed data (`'READY'` not `'ACTIVE'`)
- ☐ Fix GPU status values in seed data (`'IN_USE'` not `'ALLOCATED'`)
- ☐ Fix migration method values in seed data (`'CHECKPOINT'` not `'CRIU'` , etc.)
- ☐ Fix session backend values (`'ZELLIJ'` is not in the CHECK constraint)

Phase 3: Automation

- ☐ Add `make gen-schema` target that runs `.gen_schema.py` and fails if there's uncommitted changes
- ☐ Add pre-commit hook to run drift guard on every commit touching migrations/
- ☐ Wire drift guard into CI (when CI is re-enabled per PRR gap item 8)

Phase 4: Partition Automation

- ☐ Create a periodic job to add new `audit_log_YYYY_MM` partitions ahead of time
 - ☐ Create a retention policy to detach/drop old partitions
 - ☐ Consider `pg_partman` for automatic partition management
-

Part 7: Recommended Schema Improvements

7.1 Missing Indexes

High Priority

```
-- Composite index for the most common scheduling query:
-- "Find all READY GPU_WORKER nodes in us-east-1"
CREATE INDEX idx_nodes_role_status_region ON nodes(role,
    status, region);

-- Covering index for session listing by owner with status
  filter
CREATE INDEX idx_sessions_owner_status ON sessions(owner,
    status);

-- Reservation expiry scan (used by the lease reaper)
CREATE INDEX idx_reservations_status_expires ON
    reservations(status, expires_at);

-- Scheduling queue: priority-ordered pending requests
CREATE INDEX idx_scheduling_queue_status_priority
    ON scheduling_queue(status, priority DESC);

-- GPU availability lookup: find available GPUs by vendor/
  model
CREATE INDEX idx_gpu_devices_status_vendor_model
    ON gpu_devices(status, vendor, model);

-- Audit log time-range + severity (compliance queries)
CREATE INDEX idx_audit_log_timestamp_severity
    ON audit_log(timestamp, severity);

-- Health snapshots: time-series queries per node
CREATE INDEX idx_health_snapshots_node_id_recorded_at
    ON health_snapshots(node_id, recorded_at DESC);

-- Build artifacts: dedup by hash (avoid re-uploading
  identical artifacts)
CREATE INDEX idx_build_artifacts_hash_type
    ON build_artifacts(artifact_hash, artifact_type);
```

Medium Priority

```
-- Session termination queries
CREATE INDEX idx_sessions_terminated_at ON
  sessions(terminated_at)
  WHERE terminated_at IS NOT NULL;

-- Active sessions only (partial index)
CREATE INDEX idx_sessions_active ON sessions(node_id,
  priority)
  WHERE status = 'RUNNING';

-- Orphaned GPU detection
CREATE INDEX idx_gpu_devices_allocated ON
  gpu_devices(allocated_to)
  WHERE allocated_to IS NOT NULL;

-- Stale scheduling entries
CREATE INDEX idx_scheduling_queue_stale ON
  scheduling_queue(created_at)
  WHERE status IN ('PENDING', 'SCHEDULING');
```

7.2 Partitioning Recommendations

health_snapshots — Partition by RANGE (recorded_at)

`health_snapshots` is a high-volume time-series table. Monthly partitions would dramatically improve query performance and enable efficient retention.

```
-- Proposed: Convert health_snapshots to partitioned table
ALTER TABLE health_snapshots RENAME TO health_snapshots_old;

CREATE TABLE health_snapshots (
  id BIGSERIAL,
  node_id UUID NOT NULL REFERENCES nodes(id)
    ON DELETE CASCADE,
  overall_score INT NOT NULL,
  cpu_score INT NOT NULL,
  memory_score INT NOT NULL,
  disk_score INT NOT NULL,
  network_score INT NOT NULL,
  gpu_score INT NOT NULL,
  temperature_score INT NOT NULL,
  services_score INT NOT NULL,
  predictions JSONB NOT NULL DEFAULT '[]',
  metrics JSONB NOT NULL DEFAULT '{}',
```

```

        recorded_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
        PRIMARY KEY (id, recorded_at),
        -- (same CHECK constraints)
    ) PARTITION BY RANGE (recorded_at);

-- Monthly partitions (same pattern as audit_log)
CREATE TABLE health_snapshots_2026_06
    PARTITION OF health_snapshots FOR VALUES FROM
        ('2026-06-01') TO ('2026-07-01');
-- ... etc.

```

scheduling_queue — Partition by RANGE (created_at)

Completed scheduling entries accumulate rapidly. Partitioning would enable efficient archival.

llm_advisories — Partition by RANGE (created_at)

Similar to audit_log, advisories are append-mostly and benefit from time-based partitioning.

7.3 Data Retention Policies

Table	Retention Period	Strategy
audit_log	2 years	Detach old monthly partitions; archive to cold storage
health_snapshots	90 days	Drop old partitions; aggregate into daily summaries
scheduling_queue	30 days (completed)	DELETE completed entries older than 30 days
llm_advisories	1 year	Archive PENDING/ REJECTED/ EXPIRED after 30 days; keep APPLIED indefinitely

Table	Retention Period	Strategy
migration_history	Indefinite	Small table; keep all records
session_windows/ panes	Cascade with session	Deleted when parent session is deleted
reservations	7 days after expiry	DELETE released/expired reservations older than 7 days

Retention Job Template

```
-- Daily retention job (run via pg_cron or external scheduler)

-- 1. Detach old audit_log partitions
-- ALTER TABLE audit_log DETACH PARTITION audit_log_2024_01
--   CONCURRENTLY;

-- 2. Purge old health snapshots
DELETE FROM health_snapshots
WHERE recorded_at < NOW() - INTERVAL '90 days';

-- 3. Purge completed scheduling entries
DELETE FROM scheduling_queue
WHERE status IN ('COMPLETED', 'CANCELLED')
  AND completed_at < NOW() - INTERVAL '30 days';

-- 4. Purge expired reservations
DELETE FROM reservations
WHERE status IN ('RELEASED', 'EXPIRED')
  AND updated_at < NOW() - INTERVAL '7 days';
```

7.4 Backup and Recovery Procedures

PostgreSQL Backup

```
# Full logical backup (daily)
pg_dump -Fc -f /backup/helix_$(date +%Y%m%d).dump helix

# Schema-only backup (on migration change)
pg_dump -s -f /backup/helix_schema_$(date +%Y%m%d).sql helix
```

```
# Continuous WAL archiving (for point-in-time recovery)
# In postgresql.conf:
#   wal_level = replica
#   archive_mode = on
#   archive_command = 'cp %p /backup/wal/%f'
```

DQLite Backup

```
# DQLite supports online snapshots via its API
# On each node:
curl -X POST http://localhost:8181/1.0/backups -d '{"name":
    "daily"}'
```

SQLite Registry Backup

```
# Simple file copy (SQLite is a single file)
cp data/hxc_registry.db /backup/hxc_registry_$(date +%Y%m%d).db

# For online backup without locking:
sqlite3 data/hxc_registry.db ".backup /backup/hxc_registry_$(date +%Y%m%d).db"
```

etcd Backup

```
# Snapshot etcd data
etcdctl snapshot save /backup/etcd_$(date +%Y%m%d).snapshot

# Verify snapshot integrity
etcdctl snapshot status /backup/etcd_$(date +%Y%m%d).snapshot --write-table
```

Redis Backup

```
# Trigger RDB snapshot
redis-cli BGSAVE

# Copy the dump file
cp /var/lib/redis/dump.rdb /backup/redis_$(date +%Y%m%d).rdb
```

Recovery Procedures

Component	RPO	RTO	Recovery Method
PostgreSQL	~0 (WAL streaming)	Minutes	<code>pg_restore</code> from latest dump + WAL replay
DQLite	Seconds	Minutes	Raft log replay from surviving replicas
SQLite Registry	Daily	Minutes	File copy restore
etcd	Seconds	Minutes	<code>etcdctl snapshot restore</code> + Raft replay
Redis	Minutes	Seconds	RDB restore + AOF replay (if enabled)

7.5 Additional Recommendations

7.5.1 ENUM Types vs CHECK Constraints

The current schema uses CHECK constraints for enum validation. Consider converting high-cardinality enums to PostgreSQL ENUM types for:

- Better performance (ENUM comparison vs string comparison)
- Stronger type safety
- Automatic catalog metadata for application code generation

```
-- Example conversion:
CREATE TYPE node_status AS ENUM (
    'JOINING', 'READY', 'DRAINING', 'OFFLINE',
    'MAINTENANCE', 'EVICTED'
);
ALTER TABLE nodes ALTER COLUMN status TYPE node_status
    USING status::node_status;
```

Caveat: ENUM types cannot have values removed without a table rewrite. Use only for truly stable enumerations.

7.5.2 JSONB Schema Validation

Add check constraints or trigger-based validation for JSONB columns that have expected schemas:

```
-- Example: Validate gpu_request structure
ALTER TABLE sessions ADD CONSTRAINT
    sessions_gpu_request_schema
```

```

CHECK (
    gpu_request IS NULL
    OR (gpu_request ? 'gpu_count'
        AND (gpu_request->>'gpu_count')::int > 0)
);

```

7.5.3 Created-By / Updated-By Tracking

Add `created_by` and `updated_by` columns to key tables for full auditability:

```

ALTER TABLE sessions ADD COLUMN created_by TEXT NOT NULL
    DEFAULT 'system';
ALTER TABLE build_jobs ADD COLUMN created_by TEXT NOT NULL
    DEFAULT 'system';

```

7.5.4 Soft Delete for Critical Tables

Consider adding soft-delete support for nodes and users to enable recovery from accidental deletions:

```

ALTER TABLE nodes ADD COLUMN deleted_at TIMESTAMPTZ;
-- Queries filter: WHERE deleted_at IS NULL

```

7.5.5 Connection Pooling Configuration

```

-- Recommended PostgreSQL settings for Helix Cluster
-- In postgresql.conf:
max_connections = 200
shared_buffers = '4GB'
effective_cache_size = '12GB'
work_mem = '64MB'
maintenance_work_mem = '512MB'
checkpoint_completion_target = 0.9
wal_buffers = '16MB'
default_statistics_target = 100
random_page_cost = 1.1 -- SSD-optimized
effective_io_concurrency = 200 -- SSD-optimized

```

End of SQL Definitions Document